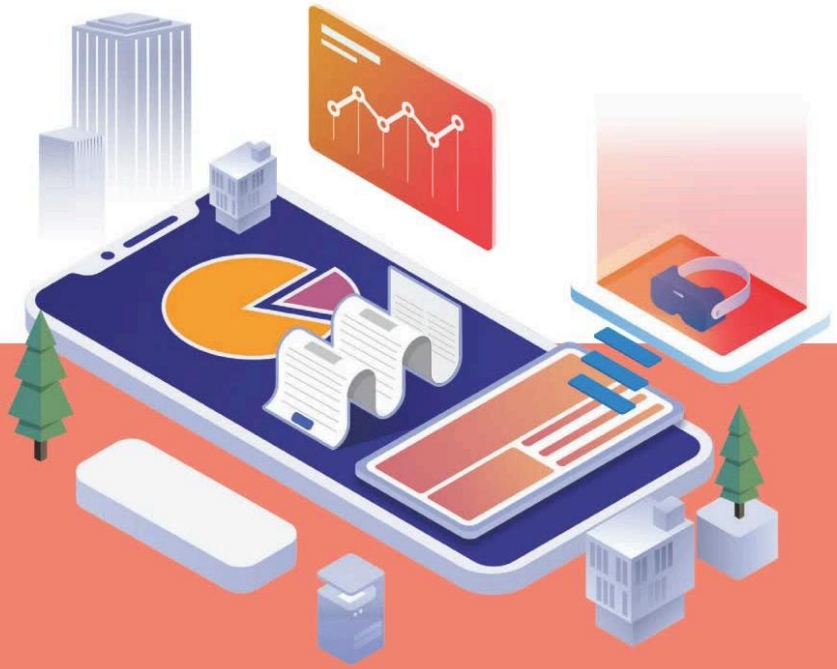
工业和信息化**精品系列**教材

Spring Boot
Enterprise Development Tutorial

Spring Boot 企业级开发教程

第 2 版



黑马程序员 编著

中国工信出版集团 人民邮电出版社 POSTS & TELECOM PRESS

工业和信息化**精品系列**教材

Spring Boot
Enterprise Development Tutorial

Spring Boot 企业级开发教程

第2版

黑马程序员 编著

人民邮电出版社
北京

图书在版编目（CIP）数据

Spring Boot企业级开发教程 / 黑马程序员编著. -- 2版. -- 北京 : 人民邮电出版社, 2024.7
工业和信息化精品系列教材
ISBN 978-7-115-63438-2

Ⅰ. ①S… Ⅱ. ①黑… Ⅲ. ①JAVA语言－程序设计－高等学校－教材 Ⅳ. ①TP312.8

中国国家版本馆CIP数据核字(2024)第000738号

内 容 提 要

本书详细讲解 Java EE 企业级开发的热门框架 Spring Boot。全书共分 10 章，其中，第 1 章和第 2 章介绍 Spring Boot 开发入门和 Spring Boot 配置；第 3～9 章介绍 Spring Boot 开发 Web 应用时常用的技术，包括 Spring Boot 的 Web 应用支持、整合 Thymeleaf、数据访问、整合缓存、安全管理、消息服务、任务调度和邮件发送；第 10 章带领读者开发一个综合项目——瑞吉外卖，希望读者通过项目实战，深刻体会 Spring Boot 框架开发 Web 应用的便捷，并能够融会贯通所学的知识。

本书配套丰富的教学资源，包括教学 PPT、源代码、教学大纲、教学设计等，为帮助初学者更好地学习本书中的内容，作者还提供在线答疑服务。

本书可作为高等教育本、专科院校计算机相关专业的教材，也可作为编程人员的自学参考书。

◆ 编　　著　黑马程序员
责任编辑　范博涛
责任印制　焦志炜
◆ 人民邮电出版社出版发行　　北京市丰台区成寿寺路 11 号
邮编　100164　　电子邮件　315@ptpress.com.cn
网址　https://www.ptpress.com.cn
北京天宇星印刷厂印刷
◆ 开本：787×1092　1/16
印张：16　　2024 年 7 月第 2 版
字数：390 千字　　2024 年 7 月北京第 1 次印刷

定价：59.80 元

读者服务热线：(010)81055256　印装质量热线：(010)81055316
反盗版热线：(010)81055315
广告经营许可证：京东市监广登字 20170147 号

专 家 委 员 会

专委会主任：黎活明

专委会成员（按姓氏笔画排序）：

万德寅（成都东软学院）
王建文（河北水利水电学院）
王琳琳（黑龙江八一农垦大学）
史　明（山西能源学院）
毕蓉蓉（哈尔滨理工大学）
李　娅（山东外国语职业技术大学）
李红军（青岛滨海学院）
李焕勤（郑州师范学院）
杨泽民（山西大同大学）
张玉霞（邯郸学院）
张劳模（河南工程学院）
胡志伟（山西农业大学）
胡晓宏（北华大学）
韩　明（石家庄学院）
楚志刚（郑州师范学院）

前 言

Spring Boot 是基于 Spring 框架研发而成的新框架，其“约定优于配置”的思想，极大简化了 Spring 应用的构建，其内部集成了大量第三方技术，使得 Spring Boot 在开发程序时变得更加便捷。近些年，微服务技术逐渐流行，Spring Boot 的很多特性非常适合微服务的开发。这些优点使得 Spring Boot 成为了使用热度很高的技术。Spring Boot 的版本在 2018 年 3 月从 1.x 升级到了 2.x，本书将重点讲解 Spring Boot 2.x 版本的相关内容。

◆ 为什么要学习本书

为加快推进党的二十大精神进教材、进课堂、进头脑，本书编者贯彻“加快建设教育强国、科技强国、人才强国”的思想对教材的编写进行策划。通过教材研讨会、师资培训等渠道，广泛调动教学经验丰富的高校教师和具有多年开发经验的技术人员共同参与教材的编写与审核工作，使本书知识的难度和案例的选取既能满足教育要求，又能满足产业发展和行业人才需求。

◆ 如何使用本书

本书共分为 10 章，下面分别对每章进行简单介绍，具体如下。

- 第 1 章主要讲解 Spring Boot 开发入门，包括 Spring Boot 概述、Spring Boot 入门案例、Spring Boot 原理解析、单元测试与热部署、Spring Boot 项目打包和运行。通过本章的学习，读者能够熟悉 Spring Boot 的基本特性及执行流程的原理等入门知识。
- 第 2 章主要讲解 Spring Boot 配置，包括全局配置文件、配置绑定、引入配置文件和定义配置类、Profile 等内容。通过本章的学习，读者能够了解 Spring Boot 配置的相关知识。
- 第 3 章主要讲解 Spring Boot 的 Web 应用支持，包括注册 Java Web 三大组件、Spring Boot 管理 Spring MVC、文件上传、异常处理。通过本章的学习，读者能够对 Spring Boot 的 Web 应用支持有所了解。
- 第 4 章主要讲解 Spring Boot 整合 Thymeleaf，包括 Spring Boot 支持的模板引擎、Thymeleaf 基础入门、图书管理案例。通过本章的学习，读者能够掌握 Thymeleaf 的常用属性和标准表达式，并实现 Spring Boot 与 Thymeleaf 的整合，以及前端页面动态显示数据。
- 第 5 章主要讲解 Spring Boot 数据访问，包括 Spring Data 概述、Spring Boot 整合 Spring Data JPA、Spring Boot 整合 MyBatis-Plus、Spring Boot 整合 Redis。通过本章的学习，读者能够掌握 Spring Boot 与各种类型数据库技术的整合，同时理解 Spring Boot 与第三方数据库技术整合的原理和过程。
- 第 6 章主要讲解 Spring Boot 整合缓存，包括 Spring Boot 默认缓存管理、Spring Boot 整合 Ehcache 缓存、Spring Boot 整合 Redis 缓存。通过本章的学习，读者能够掌握 Spring

Boot 项目中常见的整合缓存技术。

- 第 7 章主要讲解 Spring Boot 安全管理，包括安全框架概述、Spring Security 基础入门、Spring Security 认证管理、Spring Security 授权管理、Spring Security 会话管理和用户退出。通过本章的学习，读者能够对 Spring Boot 的安全管理有一定的认识，同时能够使用 Spring Boot 整合 Spring Security 框架实现安全管理。
- 第 8 章主要讲解 Spring Boot 消息服务，包括消息服务概述、RabbitMQ 快速入门、Spring Boot 与 RabbitMQ 整合实现。通过本章的学习，读者能够熟悉一些主流的消息服务中间件，并掌握 RabbitMQ 消息服务中间件的原理，同时能够使用 Spring Boot 整合 RabbitMQ 实现消息服务。
- 第 9 章主要讲解任务调度和邮件发送。通过本章的学习，读者能够在实际开发中使用 Spring Boot 实现异步任务、定时任务和邮件发送。
- 第 10 章主要讲解如何开发 Spring Boot 综合项目——瑞吉外卖。通过本章的学习，读者能够掌握 Spring Boot 项目开发的全流程，同时掌握实际开发中各种情况下的技术选择。

在学习过程中，建议读者勤思考、勤总结，并动手实践书中提供的全部案例。若在学习过程中遇到困难，建议读者不要纠结，可以先往后学习，随着学习的深入，前面难懂的地方慢慢就理解了。

◆ 致谢

本书的编写和整理工作由江苏传智播客教育科技股份有限公司完成，主要参与人员有高美云、王哲、甘金龙等，全体编写人员在本书的编写过程中付出了辛勤的汗水，在此一并表示衷心的感谢。

◆ 意见反馈

尽管编者付出了最大的努力，但书中难免还有疏漏或不妥之处，欢迎读者朋友们提出宝贵意见。读者在阅读本书时，如发现任何问题或不认同之处，可以通过电子邮件（itcast_book@vip.sina.com）与我们联系。

黑马程序员
2024 年 5 月于北京

目 录

第 1 章 Spring Boot 开发入门 1

1.1 Spring Boot 概述 1
1.1.1 Spring Boot 简介 1
1.1.2 Spring Boot 的特性 2
1.2 Spring Boot 入门案例 3
1.2.1 环境准备 3
1.2.2 使用 Spring Initializr 方式构建 Spring Boot 项目 4
1.2.3 使用 Maven 方式构建 Spring Boot 项目 8
1.3 Spring Boot 原理解析 12
1.3.1 起步依赖 12
1.3.2 自动配置 15
1.3.3 执行流程 19
1.4 单元测试与热部署 22
1.4.1 单元测试 22
1.4.2 热部署 25
1.5 Spring Boot 项目打包和运行 27
1.5.1 打包为 JAR 包并运行 27
1.5.2 打包为 WAR 包并运行 30
1.6 本章小结 33
1.7 本章习题 34

第 2 章 Spring Boot 配置 36

2.1 全局配置文件 36
2.1.1 application.properties 配置文件 37
2.1.2 application.yml 配置文件 38
2.2 配置绑定 40
2.2.1 @Value 注解 40
2.2.2 Environment 对象 42
2.2.3 @ConfigurationProperties 注解 43
2.2.4 @Value 和@Configuration Properties 对比分析 44
2.3 引入配置文件和定义配置类 45
2.3.1 引入配置文件 46
2.3.2 定义配置类 48
2.4 Profile 49
2.4.1 单一文件中配置 Profile 49
2.4.2 多文件中配置 Profile 51
2.4.3 @Profile 注解 53
2.5 本章小结 54
2.6 本章习题 54

第 3 章 Spring Boot 的 Web 应用支持 56

3.1 注册 Java Web 三大组件 56
3.1.1 使用 Spring Bean 注册 Java Web 三大组件 57
3.1.2 使用 RegistrationBean 注册 Java Web 三大组件 59
3.1.3 使用注解扫描注册 Java Web 三大组件 61
3.2 Spring Boot 管理 Spring MVC 63
3.2.1 Spring MVC 自动配置的特性 63
3.2.2 自定义 Spring MVC 配置 64
3.3 文件上传 68
3.4 异常处理 72
3.4.1 Spring Boot 异常处理自动配置原理 72
3.4.2 Spring Boot 自定义异常处理 74
3.5 本章小结 76
3.6 本章习题 77

第 4 章 Spring Boot 整合 Thymeleaf 79
4.1 Spring Boot 支持的模板引擎 79
4.2 Thymeleaf 基础入门 80
4.2.1 Thymeleaf 简介 80
4.2.2 Thymeleaf 常用属性 81
4.2.3 Thymeleaf 标准表达式 82
4.3 案例：图书管理 85
4.4 本章小结 92
4.5 本章习题 92
第 5 章 Spring Boot 数据访问 94
5.1 Spring Data 概述 95
5.2 Spring Boot 整合 Spring Data JPA 96
5.2.1 Spring Data JPA 概述 96
5.2.2 Spring Data JPA 快速入门 97
5.2.3 整合 Spring Data JPA 100
5.3 Spring Boot 整合 MyBatis-Plus 106
5.3.1 MyBatis-Plus 概述 106
5.3.2 MyBatis-Plus 快速入门 107
5.3.3 整合 MyBatis-Plus 112
5.4 Spring Boot 整合 Redis 117
5.4.1 Redis 快速入门 117
5.4.2 Spring Data Redis 快速入门 121
5.4.3 整合 Redis 123
5.5 本章小结 129
5.6 本章习题 129
第 6 章 Spring Boot 整合缓存 131
6.1 Spring Boot 默认缓存管理 131
6.1.1 Spring Boot 默认缓存方案 131
6.1.2 声明式缓存注解 132
6.1.3 声明式缓存注解的应用 136
6.2 Spring Boot 整合 Ehcache 缓存 143
6.2.1 Ehcache 概述 143
6.2.2 整合 Ehcache 145
6.3 Spring Boot 整合 Redis 缓存 148
6.4 本章小结 150
6.5 本章习题 150
第 7 章 Spring Boot 安全管理 152
7.1 安全框架概述 152
7.2 Spring Security 基础入门 154
7.2.1 Spring Security 入门案例 154
7.2.2 Spring Security 结构总览 159
7.3 Spring Security 认证管理 160
7.3.1 Spring Security 认证流程 161
7.3.2 Spring Security 自定义身份认证 162
7.4 Spring Security 授权管理 170
7.4.1 Spring Security 授权流程 171
7.4.2 Spring Security 自定义授权 171
7.4.3 动态展示菜单 179
7.5 Spring Security 会话管理和用户退出 180
7.5.1 会话管理 181
7.5.2 用户退出 182
7.6 本章小结 183
7.7 本章习题 184
第 8 章 Spring Boot 消息服务 186
8.1 消息服务概述 186
8.1.1 常用消息中间件 187
8.1.2 使用消息服务的好处 187
8.2 RabbitMQ 快速入门 190
8.2.1 RabbitMQ 简介 190
8.2.2 RabbitMQ 工作模式 192
8.2.3 RabbitMQ 的下载、安装和配置 194
8.2.4 RabbitMQ 入门案例 196
8.3 Spring Boot 与 RabbitMQ 整合实现 201
8.3.1 Spring Boot 整合 RabbitMQ 环境搭建 201
8.3.2 使用 Publish/Subscribe 模式实现消息服务 202

8.3.3 使用 Routing 模式实现消息服务 210
8.3.4 使用 Topics 模式实现消息服务 212
8.4 本章小结 214
8.5 本章习题 215
第 9 章 任务调度和邮件发送 216
9.1 任务调度 216
9.1.1 异步任务 216
9.1.2 定时任务 220
9.1.3 Quartz 任务调度 225
9.2 邮件发送 227
9.2.1 发送纯文本邮件 227
9.2.2 发送带附件和图片的邮件 230
9.2.3 发送模板邮件 233
9.3 本章小结 235
9.4 本章习题 235
第 10 章 Spring Boot 综合项目实战——瑞吉外卖 237
10.1 系统概述 237
10.1.1 系统功能介绍和技术选型 237
10.1.2 项目效果预览 239
10.2 开发环境搭建 241
10.2.1 系统开发及运行环境 241
10.2.2 数据库环境搭建 241
10.2.3 项目环境搭建 242
10.3 管理端功能模块 246
10.3.1 登录管理 246
10.3.2 分类管理 246
10.3.3 菜品管理 246
10.3.4 套餐管理 246
10.3.5 订单明细 246
10.4 用户端功能模块 247
10.4.1 用户登录 247
10.4.2 地址管理 247
10.4.3 菜品展示 247
10.4.4 购物车 247
10.4.5 下单 248
10.5 本章小结 248

第 1 章

Spring Boot 开发入门

学习目标

- ★ 了解 Spring Boot 简介，能够简述 Spring Boot 的发展过程
- ★ 了解 Spring Boot 的特性，能够简述 Spring Boot 的特性
- ★ 掌握 Spring Boot 入门案例，能够使用 Spring Initializr 方式和 Maven 方式构建 Spring Boot 项目
- ★ 熟悉 Spring Boot 原理解析，能够简述 Spring Boot 起步依赖、自动配置和执行流程的原理
- ★ 了解单元测试与热部署，能够对 Spring Boot 项目进行单元测试和热部署
- ★ 熟悉 Spring Boot 项目打包和运行，能够将 Spring Boot 项目打包为 JAR 包和 WAR 包并运行

Spring 是一个非常优秀的组件管理容器，但是为了管理容器中的组件，使用 Spring 搭建 Java EE 应用程序时往往需要进行大量的配置或注解，这些配置工作都属于项目的基础搭建，通常与业务功能无关，并且不熟悉搭建过程的人员在配置时很容易出错。为了简化 Spring 应用的搭建和配置过程，Spring Boot 应运而生。Spring Boot 是一个基于 Spring 的全新开源框架，它可以简化 Spring 应用的初始搭建和配置过程，使用更加简单，功能更加丰富。下面本章将带领大家正式进入 Spring Boot 框架的学习。

1.1 Spring Boot 概述

1.1.1 Spring Boot 简介

在 Spring Boot 框架出现之前，为了解决 Java 企业级应用开发笨重臃肿的问题，Java EE 最常用的框架是 Spring。Spring 是 2003 年兴起的轻量级的 Java 开源框架，旨在简化 Java 企业级开发。

虽然 Spring 框架是轻量级的，但它的配置却是重量级的。早期版本的 Spring 专注于 XML 配置，开发一个程序需要配置各种 XML 配置文件。随着实际生产中敏捷开发的需要，以及 Spring 注解的大量出现和功能改进，自 Spring 4.x 版本开始，项目开发已基本可以脱离 XML 配置文件进行。多数开发者也逐渐感受到了基于注解开发的便利，因此，在 Spring 中使用

注解开发逐渐占据了主流地位。与此同时，Pivotal 团队在原有 Spring 框架的基础上通过注解的方式进一步简化了 Spring 框架的使用，同时基于 Spring 框架开发了全新的 Spring Boot 框架，并于 2014 年 4 月正式推出了 Spring Boot 1.0 版本，在 2018 年 3 月又推出了 Spring Boot 2.0 版本。Spring Boot 2.x 版本在 Spring Boot 1.x 版本的基础上进行了很多功能的改进和扩展，同时进行了大量的代码重构，所以在学习开发过程中，推荐使用优化后的 Spring Boot 2.x 版本。

Spring Boot 框架本身并不提供 Spring 框架的核心特性和扩展功能，它只是使构建 Spring 应用变得简单，并且在开发过程中大量使用约定优于配置（Convention over Configuration）的思想来摆脱 Spring 框架中各种复杂的手动配置。也就是说，Spring Boot 并不是替代 Spring 框架的解决方案，而是与 Spring 框架紧密结合用于提升 Spring 开发者体验的工具。同时，Spring Boot 还集成了大量常用的第三方库配置（如 Jackson、JDBC、Redis、Mail 等），使用 Spring Boot 开发程序时，几乎是开箱即用（out-of-the-box）的，大部分的 Spring Boot 应用都只需少量配置，这一特性更能促使开发者专注于业务逻辑的实现。

另外，随着近几年微服务开发的需求增长和火爆，如何快速、简便地构建一个准生产环境的 Spring 应用成为摆在开发者面前的难题。Spring Boot 框架的出现恰好完美解决了这些问题，同时其内部还简化了许多常用的第三方库配置，使微服务开发更加便利，这也间接体现了 Spring Boot 框架的优势和学习 Spring Boot 的必要性。

Spring Boot 约定优于配置的思想，对高效开发高质量的 Spring Boot 程序至关重要。开发人员在基于 Spring Boot 开发应用时，应该秉承全局观念，遵循 Spring Boot 的相关编码规范，以降低沟通成本，提高团队的协作能力，实现高效开发便于维护的 Spring Boot 程序。

1.1.2 Spring Boot 的特性

大多数 Spring Boot 应用只需要少量的配置即可，这主要源于 Spring Boot 的特性，具体如下。

1. 可快速构建独立的 Spring 应用

构建 Spring Boot 项目时，只要根据需求选择对应的场景依赖，Spring Boot 就会自动添加该场景所需要的全部依赖并提供自动化配置，在无须额外手动添加配置的情况下可以快速构建出一个独立的 Spring 应用程序。

2. 直接嵌入 Tomcat、Jetty 和 Undertow 等 Web 容器，无须部署 WAR 文件

Spring Boot 项目内嵌了 Tomcat、Jetty 和 Undertow 等 Web 容器，在项目部署过程中无须依赖其他外部容器。Spring Boot 项目不需要像传统的 Spring 应用一样打包成 WAR 文件后再部署到 Tomcat、Jetty 和 Undertow 等 Web 容器，而是可以直接将项目打包成 JAR 文件的形式，并通过命令“ java -jar xx.jar ”运行。

3. 提供固化的“starter”依赖，简化构建配置

在 Spring Boot 项目构建过程中，无须准备各种独立的 JAR 文件，只需在构建项目时根据开发场景需求选择对应的依赖启动器“ starter ”即可。在引入的依赖启动器“ starter ”内部已经包含了对应开发场景所需的依赖，会自动下载和拉取相关 JAR 包，并且开发人员只需关注 Spring Boot 的版本，不需要关心下游依赖的版本信息。例如，在 Web 开发时，确定 Spring Boot 的版本后，只需在构建项目时选择对应的 Web 场景依赖启动器 spring-boot-starter- web，

Spring Boot 项目便会自动导入 spring-webmvc、spring-web、spring-boot-starter- tomcat 等子依赖，并自动下载和拉取 Web 开发需要的相关 JAR 包。

4. 提供了大量的自动化配置类或第三方类库

Spring Boot 充分考虑了与传统 Spring 框架以及其他第三方库融合的场景，在提供了各种场景依赖启动器的基础上，其内部还默认提供了大量的自动化配置类，例如 RedisAutoConfiguration。使用 Spring Boot 开发项目时，一旦引入了某个场景的依赖启动器，Spring Boot 内部提供的默认自动化配置类就会生效，除非开发者修改了相关默认配置，例如 Redis 地址、密码等，否则开发者无须再手动进行配置文件的配置，从而大幅减少了开发人员的工作量，提高了程序的开发效率。

5. 提供生产就绪功能

Spring Boot 提供了一些用于生产环境运行时的特性，例如指标监控、健康检查和外部化配置。其中，指标监控和健康检查可以很方便地帮助运维人员在运维期间监控项目运行情况；外部化配置可以很方便地让运维人员快速完成外部化配置和部署工作。

6. 无须生成配置代码和 XML 配置文件

Spring Boot 框架内部已经实现了与 Spring 以及其他常用第三方库的整合连接，并提供了默认最优化的整合配置，使用时基本上不需要额外生成配置代码和 XML 配置文件。在需要自定义配置的情况下，Spring Boot 更提倡使用 Java 配置类替换传统的 XML 配置方式，这样更便于查看和管理。

虽然说 Spring Boot 有很多的优点，学习 Spring Boot 入门也较为简单，但是想深入理解和学习它却有一定的难度，这是因为 Spring Boot 是在 Spring 框架的基础上推出的，所以想要弄明白 Spring Boot 的底层运行机制，需要先对 Spring 框架有一定的了解。

1.2　Spring Boot 入门案例

通过 1.1 节的学习，相信读者已经对 Spring Boot 有了初步认识，为了让读者能更直观地感受 Spring Boot 项目的创建快速、使用便捷等特点，下面通过一个入门案例演示 Spring Boot 的简单使用。

1.2.1　环境准备

任何项目开发之前通常都需要先搭建好对应的环境，Spring Boot 也不例外，Spring Boot 入门案例所需要准备的环境具体如下。

1. JDK

截至本书出版，Spring Boot 最新的正式发布版本为 Spring Boot 2.7.6，后文中如没有特殊说明，本书所使用的 Spring Boot 版本为 2.7.6。Spring Boot 2.7.6 依赖 Spring Framework 5.3.24，而运行 Spring Framework 5.3.24 需要 Java 8 及以上版本进行支撑，本书选择使用本书出版时使用较为广泛并运行较为稳定的 JDK 11。

2. 项目构建工具

Spring Boot 采用模块化设计，其模块类库管理为项目构建工具提供了构建支持。Spring Boot 2.7.6 官方声明支持的项目构建工具包括 Maven 和 Gradle，其中 Maven 的版本需要 3.5

及以上，Gradle 的版本需要 6.8.x、6.9.x 和 7.x。本书将采用 Maven 3.6.3 对 Spring Boot 进行项目构建管理。

3. 开发工具

目前市面上有很多优秀的 Java IDE 工具，其中业界较为常用有 Eclipse 和 IntelliJ IDEA。相比于 Eclipse，IntelliJ IDEA（后续简称 IDEA）的开发效率更高，并且为 Spring Boot 提供了更好的支持，因此本书选择使用 IDEA 进行项目开发。IDEA 官方提供了 Ultimate（旗舰版）和 Community（社区版）两个版本，其中 Ultimate 收费但功能丰富，Community 免费但功能有限。截至本书出版，Ultimate 最新的版本为 2022.2.2，本书选择使用 IDEA 的该版本进行项目开发。

上述工具在本书提供的资源中均可找到，关于上述工具的下载和安装过程在此就不再进行说明。

1.2.2 使用 Spring Initializr 方式构建 Spring Boot 项目

Spring Initializr 是一个可以初始化 Spring Boot 项目的工具，使用 Spring Initializr 初始化的 Spring Boot 项目包含了 Spring Boot 基本的项目结构，可以在项目初始化之前对项目所需要的依赖进行选择。使用 Spring Initializr 方式构建 Spring Boot 项目时，需要确保所在主机处于联网状态下，否则将构建失败。下面讲解如何在 IDEA 中使用 Spring Initializr 方式构建 Spring Boot 项目，并在项目中创建一个 Spring MVC 程序，具体步骤如下。

1. 构建 Spring Boot 项目

启动 IDEA，在 IDEA 欢迎界面依次选择“Project”→“New Project”创建项目，在弹出的对话框中选择左侧“Spring Initializr”选项，如图 1-1 所示。

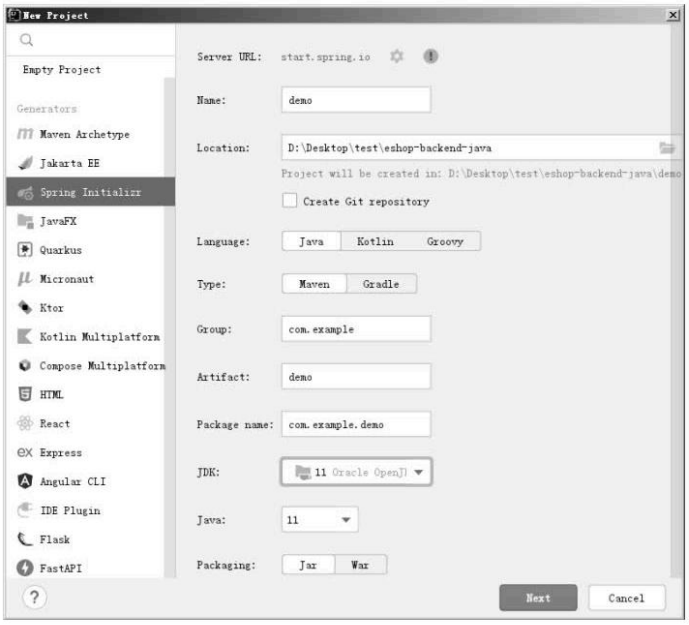


图1-1 创建项目的“Spring Initializr”选项

在图 1-1 右侧所示的“Spring Initializr”界面中，可以自定义所创建项目的相应信息，下面对这些信息进行说明，具体如下。

- Server URL：初始化 Spring Boot 项目时 Spring Initializr 工具所在的服务器地址，图 1-1 中的地址为默认地址。Server URL 可以进行修改，单击右侧的图标，会弹出设置该路径的对话框，在弹出的对话框中设置想要的 Server URL 即可。
- Name：所创建项目的名称。
- Location：所创建项目在本地存放的路径。
- Language：所创建项目使用的开发语言，可选项有 Java、Kotlin、Groovy。
- Type：使用的项目构建工具，可选项有 Maven 和 Gradle。
- Group：项目的组名，通常设置为公司或组织的反向域名。
- Artifact：项目的名称，Maven 管理项目包时用作区分的字段。
- Package name：包名，填完 Group 和 Artifact 后自动生成，保持默认选项即可。
- JDK：项目使用的 JDK。
- Java：项目使用 Java 的版本。
- Packaging：项目打包的形式，可选项有 Jar 和 War。

填写好项目的相应信息，其中项目打包方式为 Jar。单击“ Next ”按钮，进入 Spring Boot 场景依赖选择界面，如图 1-2 所示。

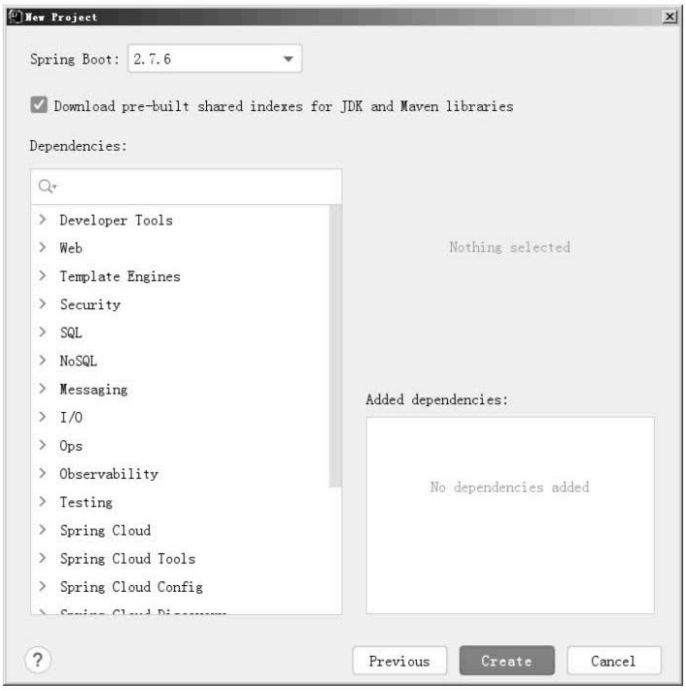


图1-2　Spring Boot 场景依赖选择界面

在图 1-2 所示界面中，可以选择项目需要使用的技术集，只需勾选对应的依赖即可。其中，在界面左上角的“ Spring Boot ”下拉框中可以选择 Spring Boot 的版本，左侧“ Dependencies ”下提供了可选择的当前主流的技术集，想要在项目中添加哪个技术框架或组件，在对应的类别中勾选对应的技术集即可。例如，想要在项目中添加 Spring MVC，可以勾选“ Web ”类别下的“ Spring Web ”，这样创建项目时就会添加 Spring Web 依赖，如图 1-3 所示。

从图 1-3 可以看出，勾选“ Spring Web ”后，在对话框右侧会显示该技术的简单说明，并在“ Added dependencies ”中自动添加 Spring Web 依赖。单击“ Create ”按钮，将根据选中的依赖创建 Spring Boot 项目，创建时会自动下载所创建项目对应的依赖。项目创建好之后的目录结构如图 1-4 所示。

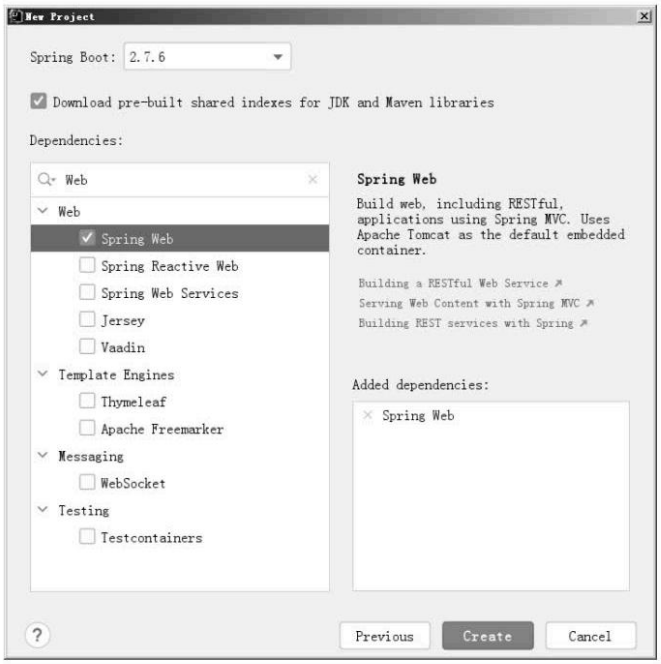


图1-3 添加Spring Web依赖

图1-4 项目的目录结构

从图 1-4 可以看出，使用 Spring Initializr 方式构建的 Spring Boot 项目中会默认生成一些文件和文件夹。其中，Chapter01Application 为项目的启动类，在该类中可以定义启动项目的方法；resources 资源文件夹下的 static 为静态资源文件夹，templates 为模板页面文件夹，application.properties 为全局配置文件；pom.xml 文件是 Maven 进行工作的主要配置文件。后续会对这些文件和文件夹的作用做进一步讲解，此处读者只需先了解项目的构成即可。

打开 Chapter01Application.java，具体如文件 1-1 所示。

文件 1-1 Chapter01Application.java

```
1 import org.springframework.boot.SpringApplication;
2 import org.springframework.boot.autoconfigure.SpringBootApplication;
3 @SpringBootApplication
4 public class Chapter01Application {
5     public static void main(String[] args) {
6         SpringApplication.run(Chapter01Application.class, args);
7     }
8 }
```

在上述代码中，Chapter01Application 类有一个@SpringBootApplication 注解进行了标注，并且只有一个 main()方法。其中，@SpringBootApplication 注解用于标注当前类是 SpringBoot 的配置类；第 6 行代码调用 SpringApplication 类的 run()方法来创建 Spring 容器，并启动具体的 Spring Boot 程序。

打开项目的 pom.xml 文件，具体如文件 1-2 所示。

文件 1-2　pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.7.6</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.itheima</groupId>
    <artifactId>chapter01</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>chapter01</name>
    <description>chapter01</description>
    <properties>
        <java.version>11</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

在上述代码中，第 7～12 行代码为当前项目继承的工程依赖。第 21～31 行代码为当前项目的依赖坐标，其中，第 22～25 行代码为 Spring Web 相关的依赖坐标；第 26～30 行代码为测试相关的依赖坐标。第 34～37 行代码为 Spring Boot 提供的 Maven 打包插件。

2. 编写 Spring MVC 控制器

在项目 com.itheima.chapter01 包下创建名称为 controller 的包，在该包下创建控制器类 HelloController，并在该类中编写处理请求的方法 index()，具体如文件 1-3 所示。

文件 1-3 HelloController.java

```
1 import org.springframework.web.bind.annotation.RequestMapping;
2 import org.springframework.web.bind.annotation.RestController;
3 @RestController
4 public class HelloController {
5     @RequestMapping("/first")
6     public String index() {
7        System.out.println("firstController is running! ");
8         return "Welcome to Spring Boot Application!";
9     }
10 }
```

在上述代码中，第 3 行代码使用@RestController 指定当前类的实例作为控制器组件；第 5 行代码使用@RequestMapping 指定 index()方法能处理的请求的路径。这些内容为 Spring MVC 的相关知识，在此就不再详细讲解。

3. 运行项目

运行文件 1-1 启动程序，此时控制台输出的信息如图 1-5 所示。

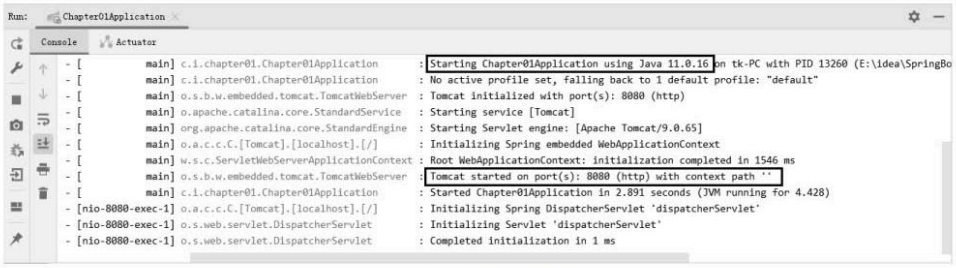


图1-5 控制台输出的信息（1）

从图 1-5 可以看出，信息提示 Chapter01Application 启动时使用 Java 11，并且 Tomcat 启动了，其端口号为 8080，上下文路径为''，即空字符串。

此时，在浏览器中访问 http://localhost:8080/first，效果如图 1-6 所示。

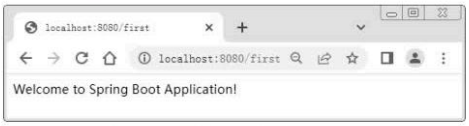


图1-6 程序测试效果（1）

图 1-6 所示的页面中显示“ Welcome to Spring Boot Application! ”，表明已在浏览器中成功发起请求，并且控制器成功将内容响应到页面。

至此 ,完成了使用 Spring Initializr 方式构建 Spring Boot 项目 ,并在项目中创建一个 Spring MVC 程序。

1.2.3 使用 Maven 方式构建 Spring Boot 项目

使用 Spring Initializr 方式构建 Spring Boot 项目非常便捷，但是如果当前主机不能联网的话 ,就无法自动下载项目所需的一些组件和依赖 ,导致这种方式无法正常使用。Spring Boot 项目可以基于 Maven 构建，如果构建项目所需要使用的组件和依赖在本地 Maven 仓库中已经存在，则无须联网也可以构建 Spring Boot 项目。下面讲解如何在 IDEA 中使用 Maven 方

式构建 Spring Boot 项目，并在项目中创建一个 Spring MVC 程序，具体步骤如下。

1. 构建 Maven 项目

启动 IDEA，在 IDEA 欢迎界面依次选择“ Project ”→“ New Project ”创建项目，在弹出的对话框中选择左侧“ Maven Archetype ”选项，如图 1-7 所示。

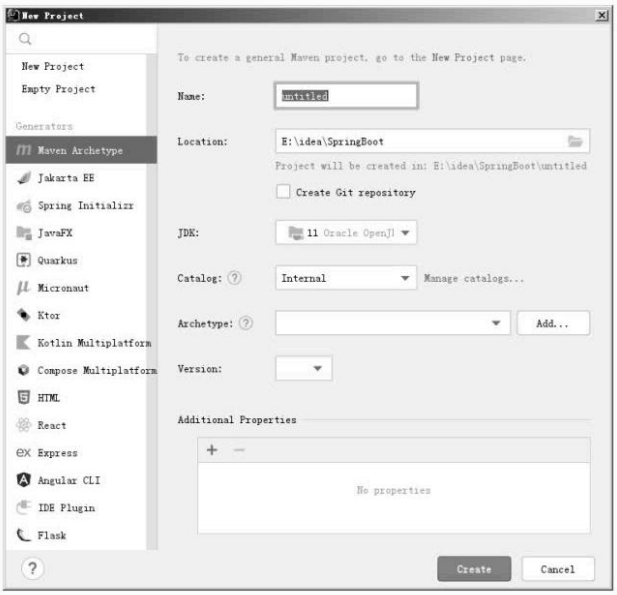


图1-7　创建项目的“ Maven Archetype ”选项

在图 1-7 所示界面右侧设置“ Maven Archetype ”选项的信息，单击界面左侧最上方的“ New Project ”可创建一个新的 Maven 项目。单击后进入“ New Project ”选项对应的界面，如图 1-8 所示。

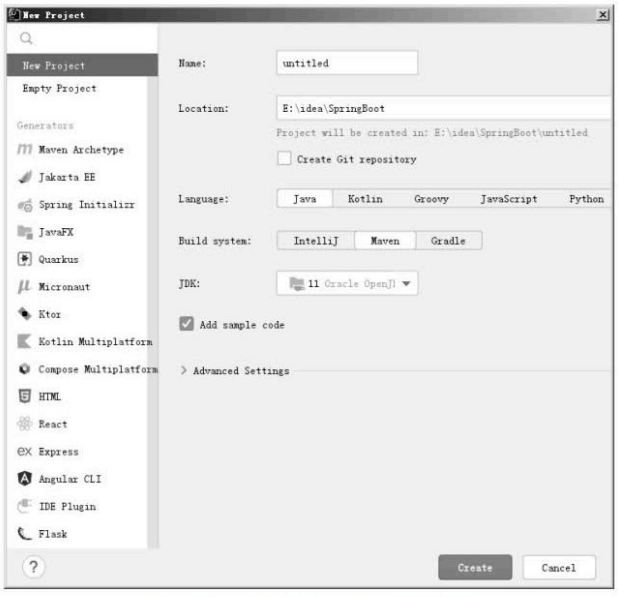


图1-8 “ New Project ”选项界面

在图 1-8 所示的界面右侧可自定义创建项目的信息，大部分与使用 Spring Initializr 方式构建 Spring Boot 项目时一致，重复的信息选项在此就不再赘述。其中，如果勾选了“ Add sample code ”，创建项目后，项目中将自动添加一些示例代码，为了保存项目的原始结构，选择不勾选。创建 Maven 项目时，单击“ Advanced Settings ”可以设置 GroupId 和 ArtifactId，其作用与使用 Spring Initializr 方式构建 Spring Boot 项目时设置 Group 和 Artifact 一样，都是 Maven 管理项目包时用作区分的字段。

设置好创建项目的信息后，单击“ Create ”按钮即可创建 Maven 项目。Maven 项目创建好之后，会默认打开项目的 pom.xml 文件。项目的结构和 pom.xml 文件的内容如图 1-9 所示。

图1-9　项目的结构和pom.xml文件的内容

从图 1-9 可以看出，创建 Maven 项目时会自动创建 java 源码文件夹和 resources 资源文件夹，但是不会在这两个文件夹下创建文件。pom.xml 文件中也没有任何的依赖信息。

2. 添加项目依赖

创建一般的 Maven 项目时，需要手动在 pom.xml 文件中插入对应的依赖信息，可以参照文件 1-2 添加项目依赖，添加的内容如下。

```
1    <parent>
2       <groupId>org.springframework.boot</groupId>
3       <artifactId>spring-boot-starter-parent</artifactId>
4       <version>2.7.6</version>
5       <relativePath/>
6    </parent>
7     <dependencies>
8       <dependency>
9          <groupId>org.springframework.boot</groupId>
10         <artifactId>spring-boot-starter-web</artifactId>
11      </dependency>
12      <dependency>
13         <groupId>org.springframework.boot</groupId>
14         <artifactId>spring-boot-starter-test</artifactId>
15         <scope>test</scope>
16      </dependency>
17   </dependencies>
```

在上述代码中，第 1～6 行代码为项目继承的工程依赖，添加该依赖后就可以使用 Spring Boot 的相关特性；第 7～17 行代码分别添加了 Spring Web 和测试相关的依赖。

小提示：

在项目 pom.xml 文件中导入新依赖或修改其他内容后，通常会自动更新而无须手动管

理。但在有些情况下，依赖文件可能还是无法自动加载，这时候就需要重新手动加载依赖文件，具体操作方法为：右键单击项目名，选择“Maven” →“Reload project”重新加载项目即可。

3. 编写程序启动类

在 java 文件夹下创建一个名称为 com.itheima.chapter01maven 的包，在该包下参照文件 1-1 创建启动类 Chapter01MavenApplication，具体如文件 1-4 所示。

文件 1-4　Chapter01MavenApplication.java

```
1 import org.springframework.boot.SpringApplication;
2 import org.springframework.boot.autoconfigure.SpringBootApplication;
3 @SpringBootApplication
4 public class Chapter01MavenApplication {
5     public static void main(String[] args) {
6         SpringApplication.run(Chapter01MavenApplication.class, args);
7     }
8 }
```

上述代码与文件 1-1 几乎一样，在此就不重复进行讲解。

4. 编写 Spring MVC 控制器

在项目 com.itheima.chapter01maven 包下创建名称为 controller 的包，在该包下创建控制器类 HelloController，该类中代码与文件 1-3 一致即可。

5. 运行项目

运行文件 1-4 启动程序，此时控制台输出的信息如图 1-10 所示。

图1-10　控制台输出的信息（2）

从图 1-10 可以看出，控制台输出的主要信息与使用 Spring Initializr 方式构建 Spring Boot 项目时几乎一致。

此时，在浏览器中访问 http://localhost:8080/first，效果如图 1-11 所示。

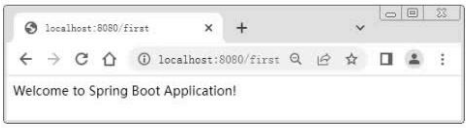


图1-11　程序测试效果（2）

图 1-11 所示的页面中显示“ Welcome to Spring Boot Application! ”，表明已在浏览器中成功发起请求，并且控制器成功将内容响应到页面。

至此，完成了使用 Maven 方式构建 Spring Boot 项目，并在项目中创建一个 Spring MVC 程序。

1.3 Spring Boot 原理解析

通过 Spring Boot 入门案例的实现，相信读者能够感受到，与使用 Spring 整合 Spring MVC 进行开发时需要设置烦琐的依赖和配置信息相比，Spring Boot 整合 Spring MVC 只需添加少量的依赖信息即可，开发过程也比较简洁。Spring Boot 项目中可以简化依赖配置和常用工程的相关配置信息，这主要依靠它的起步依赖和自动配置。下面结合 Spring Boot 入门案例对 Spring Boot 的起步依赖、自动配置和执行流程进行分析和讲解。

1.3.1 起步依赖

起步依赖本质上是一个 Maven 项目对象模型，该模型中定义了对其他库的传递依赖，Spring Boot 提供了众多起步依赖来降低项目依赖的复杂度。在 Spring Boot 入门案例中，项目的 pom.xml 文件中主要引入了两个起步依赖，分别是 spring-boot-starter-parent 和 spring-boot-starter-web，这两个依赖的相关介绍具体如下。

1. spring-boot-starter-parent 依赖

在 Spring Boot 入门案例中，项目 pom.xml 文件使用<parent>标签继承了 spring-boot-starter-parent 的依赖，其代码如下。

```
    <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>2.7.6</version>
       <relativePath/>
    </parent>
```

spring-boot-starter-parent 的版本有很多，本项目使用的版本是 2.7.6。spring-boot-starter-parent 中定义了很多常见技术的版本信息，组合成一套最优搭配的技术版本。可以在 IDEA 中按住“ Ctrl ”键单击 spring-boot-starter-parent 进入对应的源码文件中查看，其源码文件中也使用<parent>标签继承了其他依赖，具体代码如下。

```
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-dependencies</artifactId>
    <version>2.7.6</version>
  </parent>
……
```

此时，可以继续查看 spring-boot-dependencies 的源码，其源码的<properties>标签中声明了很多常见主流技术的版本号，具体如下。

```
<properties>
……
    <servlet-api.version>4.0.1</servlet-api.version>
    <slf4j.version>1.7.36</slf4j.version>
    <snakeyaml.version>1.30</snakeyaml.version>
    <solr.version>8.11.2</solr.version>
    <spring-amqp.version>2.4.8</spring-amqp.version>
    <spring-batch.version>4.3.7</spring-batch.version>
    <spring-data-bom.version>2021.2.6</spring-data-bom.version>
```

```
    <spring-framework.version>5.3.24</spring-framework.version>
    <spring-graphql.version>1.0.3</spring-graphql.version>
    <spring-hateoas.version>1.5.2</spring-hateoas.version>
    <spring-integration.version>5.5.15</spring-integration.version>
    <spring-kafka.version>2.8.11</spring-kafka.version>
    <spring-ldap.version>2.4.1</spring-ldap.version>
    <spring-restdocs.version>2.0.7.RELEASE</spring-restdocs.version>
    <spring-retry.version>1.3.4</spring-retry.version>
    <spring-security.version>5.7.5</spring-security.version>
……
</properties>
```

从上述代码中可以看出，<properties>标签中定义的其他技术的版本比较多。不同版本的 spring-boot-starter-parent 中定义的常见主流技术的版本号也可能不同，Spring Boot 对常见技术的依赖版本进行收集整理，制作出了最合理的依赖版本配置方案。

spring-boot-dependencies 对应的源码文件中使用<dependencyManagement>标签统一对常见主流技术的依赖进行声明，具体如下。

```
<dependencyManagement>
……
     <dependency>
       <groupId>org.springframework</groupId>
       <artifactId>spring-framework-bom</artifactId>
       <version>${spring-framework.version}</version>
       <type>pom</type>
       <scope>import</scope>
     </dependency>
……
</dependencyManagement>
```

从上述代码中可以看出，依赖坐标定义中没有具体的依赖版本号，而是引用了<properties>标签中定义的依赖版本属性值。这种方式只是引用坐标的依赖管理，并不直接引入依赖。如果所创建的 Spring Boot 项目继承 spring-boot-starter-parent 后，在 pom.xml 中不显示指定使用某个技术的依赖时，该技术的依赖是不会导入项目中的。所以当开发者使用某些技术时，可直接使用 Spring Boot 提供的技术，Spring Boot 可以帮助开发者对各种技术的版本进行统一管理。

2. spring-boot-starter-web 依赖

在非 Spring Boot 项目中使用某项技术时，需要先引入一些依赖坐标和设置配置信息，这些依赖和配置的内容和格式基本固定，但手动配置时非常烦琐。Spring Boot 将常用主流技术使用的固定依赖和配置进行整合，在 Spring Boot 项目中如果想要引入某个技术，只需使用 Spring Boot 针对该技术整合好的模块即可，这种模块称为 starter，也叫启动器。

Spring Boot 入门程序中引入的 spring-boot-starter-web 就是使用 Spring MVC 构建 Web 应用程序的启动器，其依赖代码如下。

```
  <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
```

进入 spring-boot-starter-web 的源码文件中查看，其源码文件中又定义了多个具体依赖

的坐标，具体如下。

```
<dependencies>
 <dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter</artifactId>
   <version>2.7.6</version>
   <scope>compile</scope>
 </dependency>
 <dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-json</artifactId>
   <version>2.7.6</version>
   <scope>compile</scope>
 </dependency>
 <dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-tomcat</artifactId>
   <version>2.7.6</version>
   <scope>compile</scope>
 </dependency>
 <dependency>
   <groupId>org.springframework</groupId>
   <artifactId>spring-web</artifactId>
   <version>5.3.24</version>
   <scope>compile</scope>
 </dependency>
 <dependency>
   <groupId>org.springframework</groupId>
   <artifactId>spring-webmvc</artifactId>
   <version>5.3.24</version>
   <scope>compile</scope>
 </dependency>
</dependencies>
```

上述代码中不仅包含了 Spring MVC 的依赖信息和 Spring 整合 Spring MVC Web 开发的依赖信息，而且包含了其他的一些依赖信息。其中，spring-boot-starter-json 是与 JSON 有关的依赖信息，JSON 是 Spring MVC 开发通常会使用的技术；spring-boot-starter-tomcat 是 Tomcat 的依赖信息。由此可知，使用启动器可以帮助开发者快速配置依赖关系。

Spring Boot 官方提供了大量的启动器，其名称基本都是以“ spring-boot-starter-技术名称 ”的格式命名的 ,通过启动器的名称通常可以知道它所提供的功能 ,例如 ,spring-boot-starter-web 表示提供 Web 相关的功能，spring-boot-starter-jdbc 表示提供 JDBC 相关的功能。常见的 Spring Boot 应用程序启动器如表 1-1 所示。

表 1-1　常见的 Spring Boot 应用程序启动器

名称	描述
spring-boot-starter-parent	核心启动器，包括自动配置支持、日志记录和 YAML，常被作为父依赖
spring-boot-starter-logging	提供 Logging 相关的日志功能
spring-boot-starter-thymeleaf	使用 Thymeleaf 视图构造 MVC Web 应用程序的启动器

续表

名称	描述
spring-boot-starter-web	使用 Spring MVC 构建 Web，包括 RESTful 应用程序，使用 Tomcat 作为默认的嵌入式容器的启动器
spring-boot-starter-test	支持常规的测试依赖，包括 Junit、Hamcrest、Mockito 和 spring-test 模块
spring-boot-starter-jdbc	结合 JDBC 和 HikariCP 连接池的启动器，对数据源自动装配，并提供 JdbcTemplate 简化数据库操作
spring-boot-starter-data-jpa	使用 Spring JPA 与 Hibernate 的启动器
spring-boot-starter-data-redis	Redis key-value 数据存储和 Spring Data Redis 与 Jedis 客户端的启动器
spring-boot-starter-log4j2	提供 Log4j2 相关的日志功能
spring-boot-starter-mail	提供邮件相关功能
spring-boot-starter-activemq	使用 Apache ActiveMQ 的 JMS 启动器
spring-boot-starter-data-mongodb	使用 MongoDB 面向文档的数据库和 Spring Data MongoDB 的启动器
spring-boot-starter-actuator	提供应用监控与监控相关的功能
spring-boot-starter-security	使用 Spring Security 的启动器
spring-boot-starter-dubbo	提供 Dubbo 框架的相关功能

表 1-1 中列出了 Spring Boot 官方提供的部分启动器，这些启动器适用于不同的场景开发，使用时只需要在 pom.xml 文件中导入相应的启动器依赖即可。

需要说明的是，Spring Boot 官方并没有为所有场景开发的技术框架都提供了启动器，例如，数据库操作框架 MyBatis、阿里巴巴的 Druid 数据源等，Spring Boot 官方就没有提供对应的启动器。为了充分利用 Spring Boot 框架的优势，一些第三方技术厂商主动与 Spring Boot 框架进行了整合，实现了各自的依赖启动器，例如，MyBatis 提供的启动器 mybatis-spring-boot-starter。不过在项目 pom.xml 文件中引入这些第三方的启动器时，需要自行配置对应的依赖版本号。

spring-boot-starter-parent 和普通的 starter 都使 Spring Boot 项目简化了配置，但是它们两个的功能却不相同。spring-boot-starter-parent 中定义了很多个常见组件或框架的依赖版本号，组合成一套最优搭配的技术版本，更便于统一管理依赖的版本，且减少了依赖的冲突。而普通的 starter 是在一个坐标中定了若干个坐标，反而减少了依赖配置的代码量。

1.3.2　自动配置

Spring Boot 采用约定大于配置的设计思想，将 Spring Boot 开发过程中可能会遇到的配置信息提前配置好，写在自动配置的 JAR 包中。项目启动时会自动检测项目类路径下所有的依赖 JAR 包，将检测到的 Bean 注册到 Spring 容器中，并根据检测的依赖进行自动配置。在之前的 Spring Boot 入门案例中并没有手动配置 Spring 的相关信息，程序启动后还会创建 Spring 的 IoC 容器吗？答案是肯定的，Spring Boot 并不是对 Spring 功能的增强，而是提供了一种快速使用 Spring 的方式，当运行项目的启动类后就会产生一个 Spring 容器对象。

Spring Boot 的启动类和其他类的主要区别是启动类上有一个@SpringBootApplication 注解，而 Spring Boot 实现自动配置的主要核心就在 @SpringBootApplication 这个注解上。查看@SpringBootApplication 的源码，其主要代码如下。

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(
    excludeFilters = {@Filter(
    type = FilterType.CUSTOM,
    classes = {TypeExcludeFilter.class}
), @Filter(
    type = FilterType.CUSTOM,
    classes = {AutoConfigurationExcludeFilter.class}
)}
)
public @interface SpringBootApplication {
……
}
```

从上述代码可以看出，@SpringBootApplication 注解内部包含了多个注解，是一个复合注解，其中@SpringBootConfiguration、@EnableAutoConfiguration、@ComponentScan 是其核心的注解，下面分别对这 3 个注解进行讲解说明。

1. @SpringBootConfiguration

@SpringBootConfiguration 标注当前类是一个配置类，它是一个复合注解，查看@SpringBootConfiguration 的源码文件，具体内容如下。

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Configuration
@Indexed
public @interface SpringBootConfiguration {
    @AliasFor(
        annotation = Configuration.class
    )
    boolean proxyBeanMethods() default true;
}
```

从上述代码可以看出，@SpringBootConfiguration 是@Configuration 的派生注解，拥有@Configuration 注解的功能，而@Configuration 是@Component 的派生注解，所以被@SpringBootApplication 标注的类可以被扫描到 Spring 的 IoC 容器中。

2. @EnableAutoConfiguration

@EnableAutoConfiguration 可以开启自动配置，它也是一个复合注解，查看@EnableAutoConfiguration 的源码文件，具体内容如下。

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
```

```
@Documented
@Inherited
@AutoConfigurationPackage
@Import({AutoConfigurationImportSelector.class})
public @interface EnableAutoConfiguration {
   String ENABLED_OVERRIDE_PROPERTY =
         "spring.boot.enableautoconfiguration";
    Class<?>[] exclude() default {};
    String[] excludeName() default {};
}
```

@EnableAutoConfiguration 主要通过上述代码中的@AutoConfigurationPackage 和@Import ({AutoConfigurationImportSelector.class})实现自动配置。下面分别对这两个注解进行讲解。

（1）@AutoConfigurationPackage

进一步查看@AutoConfigurationPackage 的源码文件，具体内容如下。

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@Import({AutoConfigurationPackages.Registrar.class})
public @interface AutoConfigurationPackage {
    String[] basePackages() default {};
    Class<?>[] basePackageClasses() default {};
}
```

从上述代码可以看出，@AutoConfigurationPackage 也是一个复合注解，其中@Import 导入了 AutoConfigurationPackages.Registrar 类，Registrar 类属于 AutoConfigurationPackages 类中的静态内部类，该类中提供了批量注册组件到 Spring 容器的方法。查看 Registrar 类源码信息，具体内容如下。

```
    static class Registrar implements ImportBeanDefinitionRegistrar,
 DeterminableImports {
       Registrar() {
       }
       public void registerBeanDefinitions(AnnotationMetadata metadata,
 BeanDefinitionRegistry registry) {
          AutoConfigurationPackages.register(registry, (String[])(
new PackageImports(metadata)).getPackageNames().
toArray(new String[0]));
       }
       public Set<Object> determineImports(AnnotationMetadata metadata) {
          return Collections.singleton(new PackageImports(metadata));
       }
    }
```

在上述代码中，Registrar 类的 registerBeanDefinitions()方法主要用于将组件注册到 Spring 容器中，注册的组件为被@SpringBootApplication 标注的启动类下所有包及子包里扫描到的 Bean。

通过对@AutoConfigurationPackage 源代码的追踪，可以发现@AutoConfigurationPackage 的主要作用是获取项目启动类所在根目录，从而指定组件扫描器扫描的包位置。因此，在定义项目包结构时，项目启动类要定义在最外层的根目录位置，然后在根目录位置内部建

立子包和类进行业务开发，这样定义的类才能够被组件扫描器扫描到。

（2）@Import({AutoConfigurationImportSelector.class})

通过@Import({AutoConfigurationImportSelector.class}导入 AutoConfigurationImportSelector 类，AutoConfigurationImportSelector 类中提供了一个获取自动配置的方法 getAutoConfigurationEntry()，用于筛选出当前项目环境需要启动的自动配置类，从而实现当前项目运行所需的自动配置环境。查看 AutoConfigurationImportSelector 类的 getAutoConfigurationEntry()方法，具体代码如下。

```
1  protected AutoConfigurationEntry getAutoConfigurationEntry(
2      AnnotationMetadata annotationMetadata) {
3      if (!this.isEnabled(annotationMetadata)) {
4          return EMPTY_ENTRY;
5      } else {
6          AnnotationAttributes attributes =
7              this.getAttributes(annotationMetadata);
8          List<String> configurations =
9           this.getCandidateConfigurations(annotationMetadata, attributes);
10         configurations = this.removeDuplicates(configurations);
11         Set<String> exclusions =
12             this.getExclusions(annotationMetadata, attributes);
13         this.checkExcludedClasses(configurations, exclusions);
14         configurations.removeAll(exclusions);
15         configurations =
16             this.getConfigurationClassFilter().filter(configurations);
17        this.fireAutoConfigurationImportEvents(configurations,exclusions);
18        return new AutoConfigurationEntry(configurations, exclusions);
19     }
20 }
```

在上述代码中，第 3 行代码判断传入的元注解的值是否可用，程序执行时传入的元注解的值为@SpringBootApplication 所标注的类的全类名。如果该值为不可用则执行第 4 行代码返回一个空的自动配置实体；如果可用，则执行第 5～19 行 else 代码块中的代码。其中，第 8～9 行代码调用 getCandidateConfigurations()方法，从 Spring Boot 自动配置 JAR 包的 ME TA-INF/spring.factories 和 META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports 文件中获取所有候选自动配置类，其中后者是 Spring Boot 2.7 开始采用的自动配置文件，为了兼容之前的版本，前者也可以继续使用。第 16 行代码表示配置类的过滤器调用 filter()方法，该方法的作用是对所有候选的自动配置类进行筛选，根据项目 pom.xml 文件中加入的依赖筛选出最终符合当前项目运行环境的自动配置类。

META-INF/spring.factories 和 META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports 文件中的信息如图 1-12 和图 1-13 所示。

图 1-12 所示的 spring.factories 文件是以键值对的形式存放自动配置类的信息，图 1-13 所示的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 文件中每行存放一个自动配置类名。

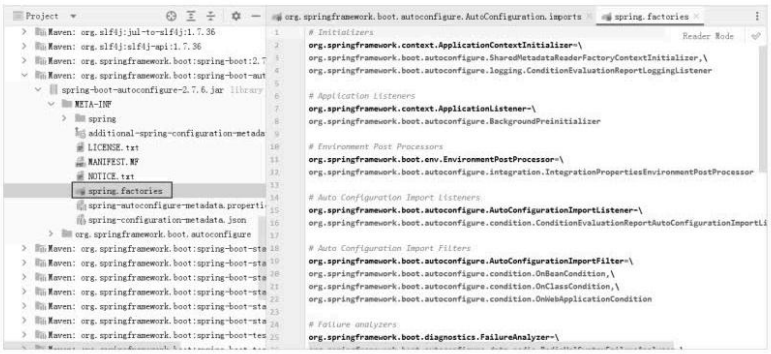


图1 - 12　spring.factories

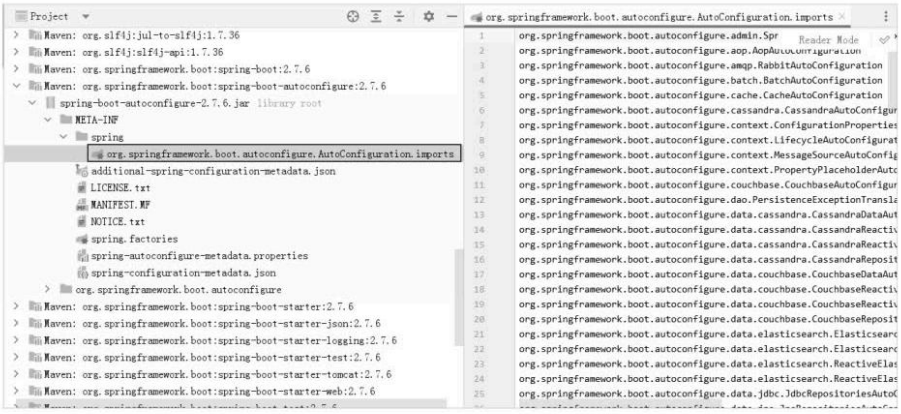


图1 - 13　org.springframework.boot.autoconfigure.AutoConfiguration.imports

3. @ComponentScan

@ComponentScan 注解是一个组件包扫描器，其主要作用是扫描指定包及其子包下所有注解类文件，并将其作为 Spring 容器的组件使用。@ComponentScan 注解具体扫描的包的根路径由 Spring Boot 项目主程序启动类所在包的位置决定，在扫描过程中由前面介绍的@AutoConfigurationPackage 注解进行解析，从而得到 Spring Boot 项目主程序启动类所在包的具体位置。

@ComponentScan 中使用 excludeFilters 属性指定扫描时需要过滤掉的不加载到 Spring 容器的类，使用 TypeExcludeFilter 类和 AutoConfigurationExcludeFilter 类指定过滤规则。其中，TypeExcludeFilter 类用于在 BeanFactory（可以理解为 Spring IoC 容器）中查找所有类型为 TypeExcludeFilter 的组件，并执行其自定义的过滤方法；AutoConfigurationExcludeFilter 用于过滤其他同时使用@Configuration 和@EnableAutoConfigure 的类。

1.3.3　执行流程

Spring Boot 入门案例中启动类的 main()方法只有一行代码，即使用 SpringApplication 类调用 run()方法，执行该行代码时，就启动了整个 Spring Boot 项目。为了进一步了解 Spring Boot 的启动原理，下面基于 Spring Boot 入门案例讲解 Spring Boot 的执行流程。

查看启动类中 SpringApplication 调用的 run()方法的源码，核心代码具体如下。

```
public static ConfigurableApplicationContext run(Class<?> primarySource,
String... args) {
```

```
    return run(new Class[]{primarySource}, args);
}
public static ConfigurableApplicationContext run(Class<?>[] primarySources,
String[] args) {
    return (new SpringApplication(primarySources)).run(args);
}
```

从上述代码可以看出，SpringApplication 的 run()方法中调用了另一个重载的 run()方法，被调用的重载 run()方法内部执行了两个操作 ,分别是创建 SpringApplication 实例和调用 run()方法，下面对这两个操作的实现进行讲解，具体如下。

1. 创建 SpringApplication 实例

创建 SpringApplication 实例时会初始化项目启动所需的资源和对象，查看 SpringApplication 类的构造方法，具体代码如下。

```
1  public SpringApplication(Class<?>... primarySources) {
2      this((ResourceLoader)null, primarySources);
3  }
4  public SpringApplication(ResourceLoader resourceLoader,
5     Class<?>... primarySources) {
6      this.sources = new LinkedHashSet();
7      this.bannerMode = Mode.CONSOLE;
8      this.logStartupInfo = true;
9      this.addCommandLineProperties = true;
10     this.addConversionService = true;
11     this.headless = true;
12     this.registerShutdownHook = true;
13     this.additionalProfiles = Collections.emptySet();
14     this.isCustomEnvironment = false;
15     this.lazyInitialization = false;
16     this.applicationContextFactory = ApplicationContextFactory.DEFAULT;
17     this.applicationStartup = ApplicationStartup.DEFAULT;
18     this.resourceLoader = resourceLoader;
19     Assert.notNull(primarySources, "PrimarySources must not be null");
20     this.primarySources =
21         new LinkedHashSet(Arrays.asList(primarySources));
22     this.webApplicationType = WebApplicationType.deduceFromClasspath();
23     this.bootstrapRegistryInitializers = new ArrayList(this.
24        getSpringFactoriesInstances(BootstrapRegistryInitializer.class));
25     this.setInitializers(this.getSpringFactoriesInstances(
26        ApplicationContextInitializer.class));
27     this.setListeners(this.getSpringFactoriesInstances(
28        ApplicationListener.class));
29     this.mainApplicationClass = this.deduceMainApplicationClass();
30 }
```

在上述代码中，第 2 行代码调用了第 4～30 行代码所示的重载的构造方法。第 4～30 行代码所示的构造方法中对属性进行了初始化。

第 18 行代码将传入的资源加载器设置为项目的资源加载器 ,传入的资源加载器此时为 null，资源加载器是用于加载类路径或文件系统资源等资源的策略接口。

第 20～21 行代码将启动类转换为 ArrayList，最后放到 LinkedHashSet 集合中赋值给

primarySources 属性。

第 22 行代码根据 ClassPath 推导出当前启动的项目的类型，项目的类型有以下 3 种。

- NONE：该应用程序不应作为 Web 应用程序运行，也不应启动嵌入式 Web 服务器。
- SERVLET：该应用程序应作为基于 Servlet 的 Web 应用程序运行，并应启动嵌入式 Servlet Web 服务器。
- REACTIVE：该应用程序应作为响应式 Web 应用程序运行，并应启动嵌入式响应式 Web 服务器。

由于 Spring Boot 入门案例引入了 spring-boot-starter-web 依赖，其依赖中包含推导时的 WEBMVC_ INDICATOR_CLASS 类，所以会自动选择 SERVLET 类型。

第 23～28 行代码依次调用 getSpringFactoriesInstances()方法从项目的所有 JAR 包中的 META-INF/spring.factories 文件加载和实例化 BootstrapRegistryInitializer、ApplicationContext Initializer 和 ApplicationListener 的实现类，这 3 个接口的说明如下。

- BootstrapRegistryInitializer：是在使用 BootstrapRegistry 之前对其进行初始化的回调接口，BootstrapRegistry 是为了让 Spring Boot 在 ApplicationContext 准备好之前能有机会注册一些类，并且通过触发 ApplicationReadyEvent 来处理一些逻辑。
- ApplicationContextInitializer：是用于在刷新之前初始化 Spring Configurable Application Context 的回调接口。
- ApplicationListener：是由应用程序事件监听器实现的接口，也是基于观察者设计模式的标准 java.util.EventListener 接口。

第 29 行代码用于获取当前运行的 main()方法所在的类，设置项目 main()方法启动的主程序启动类。

从上述初始化过程可以得出，创建 SpringApplication 实例主要执行了以下操作。

① 初始化资源加载器，此处将资源加载器置空。
② 初始化加载资源类集合。
③ 推断当前 Web 应用类型
④ 设置应用上下文初始化器。
⑤ 设置监听器。
⑥ 推断主应用类。

2. 调用 run()方法

分析完(new SpringApplication(primarySources)).run(args)源码前一部分 SpringApplication 实例对象的初始化创建后，查看 run()方法执行的过程，run()方法的核心代码具体如下。

```
1  public ConfigurableApplicationContext run(String... args) {
2      long startTime = System.nanoTime();
3      DefaultBootstrapContext bootstrapContext =
4          this.createBootstrapContext();
5      ConfigurableApplicationContext context = null;
6      this.configureHeadlessProperty();
7      SpringApplicationRunListeners listeners = this.getRunListeners(args);
8      listeners.starting(bootstrapContext, this.mainApplicationClass);
9      try {
10         ApplicationArguments applicationArguments =
11             new DefaultApplicationArguments(args);
```

```
12          ConfigurableEnvironment environment = this.prepareEnvironment(
13              listeners, bootstrapContext, applicationArguments);
14          this.configureIgnoreBeanInfo(environment);
15          Banner printedBanner = this.printBanner(environment);
16          context = this.createApplicationContext();
17          context.setApplicationStartup(this.applicationStartup);
18          this.prepareContext(bootstrapContext, context, environment,
19              listeners, applicationArguments, printedBanner);
20          this.refreshContext(context);
21          this.afterRefresh(context, applicationArguments);
22          Duration timeTakenToStartup = Duration.ofNanos(System.nanoTime() -
23              startTime);
24          if (this.logStartupInfo) {
25              (new StartupInfoLogger(this.mainApplicationClass)).
26                  logStarted(this.getApplicationLog(), timeTakenToStartup);
27          }
28          listeners.started(context, timeTakenToStartup);
29          this.callRunners(context, applicationArguments);
30      } catch (Throwable var12) {
31          this.handleRunFailure(context, var12, listeners);
32          throw new IllegalStateException(var12);
33      }
34      ……
35  }
```

在上述代码中，第 3～4 行代码创建启动上下文对象；第 7～8 行代码获取并启动运行监听器；第 10～11 行代码将应用程序参数封装为 ApplicationArguments 类型对象；第 12～13 行代码配置环境信息 ConfigurableEnvironment 的基础信息，所配置的基础信息为上下文启动时可能需要配置的信息；第 14 行代码配置需要忽略的 Bean 信息；第 16～17 行代码创建应用程序上下文对象，并设置上下文对象的应用程序启动器；第 18 行代码对应用上下文环境进行准备；第 20 行代码刷新应用程序上下文，初始化 IoC 容器里面的 Bean；第 28 行代码调用了监听器的 started()方法，通知监听器上下文启动完成。

至此，有关 Spring Boot 执行流程的内容已分析完毕。通过对项目启动过程中两阶段源码的详细分析，相信读者对 Spring Boot 执行流程已经有了大体的认识。虽然大部分内容都较为复杂，但通过对一些核心技术的源码进行学习可以提升自身技术水平。软件开发人员在日常的开发和学习过程中，需要保持勇于探究的精神，不畏困难，以坚持不懈的探索精神，积极寻求有效的问题解决方法，不断提升自我能力和韧性。

1.4 单元测试与热部署

1.4.1 单元测试

单元测试是针对一个独立的工作单元进行正确性验证的测试，对程序开发来说非常重要，通过单元测试不仅能增强程序的健壮性，而且为程序的重构提供了依据。Spring Boot 为项目的单元测试提供了很好的支持。在使用时，需要提前在项目的 pom.xml 文件中添加 spring-boot-starter-test 测试依赖启动器，具体代码如下。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

在上述代码中，展示了 Spring Boot 框架集成单元测试提供的依赖启动器，其<scope>范围默认为 test。

在项目中添加测试依赖启动器后，可以编写相关测试代码对 Spring Boot 项目中相关功能进行单元测试。根据测试时是否需要启动 Web 服务器，可以将单元测试分为 Web 环境模拟测试和业务组件测试，下面对这两类测试分别进行讲解。

1. Web 环境模拟测试

进行 Web 环境模拟测试包含启动 Web 环境和发送 Web 请求两个部分，下面通过案例演示 Web 环境模拟测试。

首先，启动 Web 环境。Spring Boot 提供了@SpringBootTest 用于修饰单元测试用例类。通过@SpringBootTest 注解的 webEnvironment 属性可以设置在测试用例中启动 Web 环境。webEnvironment 属性可以设置的值有以下 4 个。

- MOCK：为 webEnvironment 属性的默认值，会加载一个 WebApplicationContext 并提供一个模拟 Servlet 环境，属于适配性的配置，根据当前的设置确认是否启动 Web 环境，例如，使用了 Servlet 的 API 就会启动 Web 环境。
- DEFINED_PORT：加载一个 EmbeddedWebApplicationContext 并提供一个真正的 Servlet 环境，使用自定义的端口作为 Web 服务器端口。
- RANDOM_PORT：加载一个 EmbeddedWebApplicationContext 并提供一个真正的 Servlet 环境，使用随机端口作为 Web 服务器端口。
- NONE：使用 SpringApplication 加载 ApplicationContext，但不启动 Web 环境。

在项目 chapter01 测试文件夹下的 com.itheima.chapter01 包下编写测试用例类 WebTest，在该类中启动 Web 环境，并使用随机端口作为 Web 服务器端口，具体如文件 1-5 所示。

文件 1-5　WebTest.java

```
1  import org.springframework.boot.test.context.SpringBootTest;
2  @SpringBootTest(webEnvironment =
3      SpringBootTest.WebEnvironment.RANDOM_PORT)
4  public class WebTest {
5  }
```

接着，发起 Web 请求。在 Web 环境模拟测试发起 Web 请求时，需要先开启 Web 虚拟调用功能。在测试用例类上使用@AutoConfigureMockMvc 注解开启 Web 虚拟调用功能。在文件 1-5 中定义测试方法，在测试方法中发送 Web 请求，具体代码如下。

```
1  import org.junit.jupiter.api.Test;
2  import org.springframework.beans.factory.annotation.Autowired;
3  import org.springframework.boot.test.autoconfigure.web.servlet.
4         AutoConfigureMockMvc;
5  import org.springframework.boot.test.context.SpringBootTest;
6  import org.springframework.test.web.servlet.MockMvc;
7  import org.springframework.test.web.servlet.
8          request.MockHttpServletRequestBuilder;
9  import org.springframework.test.web.servlet.
```

```
10        request.MockMvcRequestBuilders;
11 @SpringBootTest(webEnvironment =
12     SpringBootTest.WebEnvironment.RANDOM_PORT)
13 @AutoConfigureMockMvc
14 public class WebTest {
15    @Test
16    void testWeb(@Autowired MockMvc mvc) throws Exception {
17        //创建虚拟请求，当前访问/first
18        MockHttpServletRequestBuilder builder =
19             MockMvcRequestBuilders.get("/first");
20        //执行对应的请求
21        mvc.perform(builder);
22    }
23 }
```

在上述代码中，第 16 行代码用于定义发起虚拟调用的对象 MockMvc，并通过自动装配的形式初始化对象；第 18～19 行代码创建了一个虚拟请求对象，并封装了请求的路径；第 21 行代码使用 MockMvc 对象执行对应的请求。

最后，选中单元测试方法 testWeb()，右键单击“ Run'testWeb()' ”选项启动测试方法，此时控制台的打印信息如图 1-14 所示。

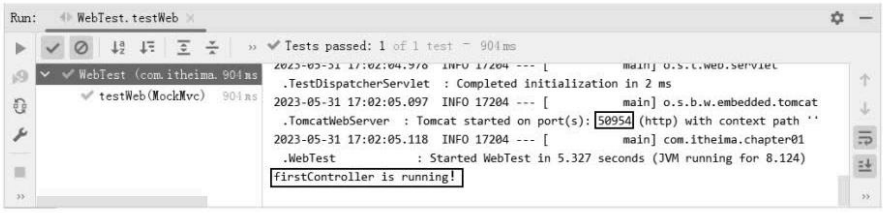


图1-14 文件1-5运行结果

从图 1-14 中可以看出，启动的 Tomcat 端口号不是默认的 8080，而是 50954，并且输出了“ firstController is running !”。说明测试用例中成功模拟出了 Web 环境，并成功发送了 Web 请求，实现了 Web 环境模拟测试。

2. 业务组件测试

当只需测试 Service 层或数据访问对象（Data Access Object，DAO）层等业务组件时，不需要启动 Web 服务器，测试方法会直接调用被测试组件的方法。

首先，在项目 chapter01 的 com.itheima.chapter01 包下创建名称为 service 的包，在该包下创建类 HelloService，具体如文件 1-6 所示。

文件 1-6 HelloService.java

```
1 import org.springframework.stereotype.Service;
2 @Service
3 public class HelloService {
4    public void getById(Integer id){
5        System.out.println("Service get id:"+id);
6    }
7 }
```

在上述代码中，第 2 行代码标记当前类是一个 Service 类，程序启动时会将当前类自动注入到 Spring 容器中。

接着，定义测试用例类。在项目 chapter01 测试文件夹下的 com.itheima.chapter01 包下编写测试用例类 ServiceTest，在该类中不启动 Web 环境，具体如文件 1-7 所示。

文件 1-7　ServiceTest.java

```
1  import com.itheima.chapter01.service.HelloService;
2  import org.junit.jupiter.api.Test;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.boot.test.context.SpringBootTest;
5  @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE)
6  public class ServiceTest {
7      @Autowired
8      private HelloService helloService;
9      @Test
10     void testService() {
11         helloService.getById(13);
12     }
13 }
```

在上述代码中，第 5 行代码 webEnvironment 属性中指定不启动 Web 环境；第 7～8 行代码使用了@Autowired 注解将容器中类型为 HelloService 的 Bean 注入 helloService 实例变量中；第 11 行代码调用 HelloService 实例的 getById()方法。

最后，选中单元测试方法 testService()，右键单击“ Run'testService()' ”选项启动测试方法，此时控制台的打印信息如图 1-15 所示。

图1-15　文件1-7运行结果

在图 1-15 中，控制台没有输出 Tomcat 的相关启动信息，但输出了“ Service get id:13 ”的信息，说明成功调用了 HelloService 实例的 getById()方法，业务组件测试成功。

1.4.2　热部署

在开发过程中，可能会不断地修改业务代码，每次修改之后想要测试最新的效果往往需要重启服务，这种重复的启动操作极大地降低了程序开发效率。热部署是指不用重启服务，服务器会自己悄悄地把更新后的程序重新加载一遍。为了提高开发效率，Spring Boot 框架提供了热部署的依赖，基于该依赖可以对项目进行热部署。下面在 chapter01 项目基础上讲解如何进行热部署，具体步骤如下。

1. 添加热部署依赖

在 Spring Boot 项目进行热部署测试之前，需要先在项目的 pom.xml 文件中添加热部署依赖，具体代码如下。

```
<!-- 引入热部署依赖 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
```

```
</dependency>
```

2. 设置启动热部署

在 IDEA 的菜单栏中依次选择“ File ”→“ Settings ”，进入 IDEA 的设置对话框，然后选择“ Build，Execution，Deployment ”的“ Compiler ”选项，如图 1-16 所示。在右侧勾选“ Build project automatically ”选项将项目设置为自动编译，然后单击“ Apply ”“ OK ”按钮保存设置。

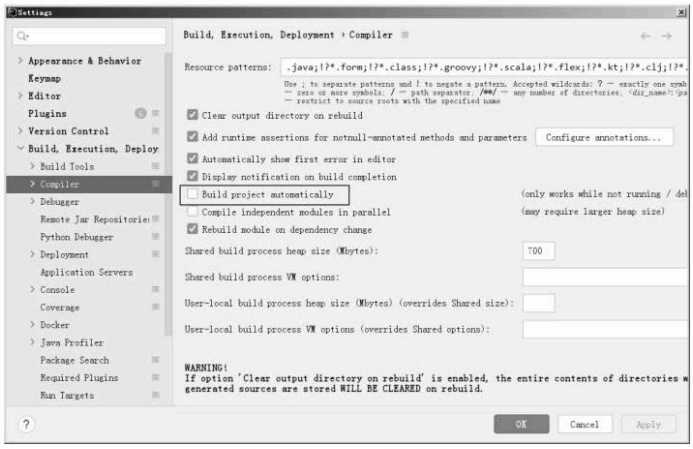


图1-16 Compiler选项

在 IDEA 的设置对话框中选择“ Advanced Settings ”选项，如图 1-17 所示。在右侧勾选“ Compiler ”下的“ Allow auto-make to start even if developed application is currently running ”选项，允许自动启动当前正在运行的应用程序，然后单击“ Apply ”“ OK ”按钮保存设置。

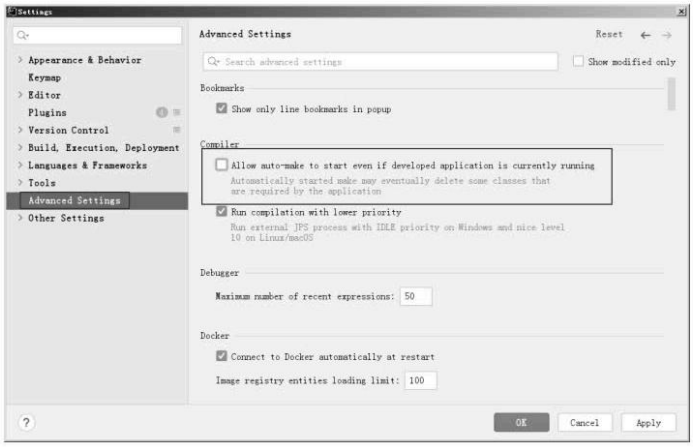


图1-17 Advanced Settings选项

3. 热部署效果测试

启动 chapter01 项目，在浏览器中访问“ http://localhost:8080/first ”，如图 1-18 所示。

从图 1-18 可以看出，页面原始输出的内容是“ Welcome to Spring Boot Application! ”。

为了测试配置的热部署是否有效，接下来，在不关闭当前项目的情况下，将 HelloController 类中 index()方法的返回值修改为“ Welcome to Spring Boot Application!---developed ”。5 秒后刷新图 1-18 所示的浏览器页面，如图 1-19 所示。

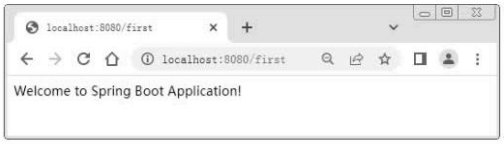


图1-18　程序测试效果

图1-19　热部署后的访问效果

从图 1-19 可以看出，浏览器输出了“ Welcome to Spring Boot Application!---developed ”，说明项目热部署配置成功。需要在更新后 5 秒再刷新，是因为 IDEA 设置了当 IDEA 失去焦点 5 秒后才进行热部署，以避免修改项目时每敲一个字母服务器就重新构建一次。

1.5　Spring Boot 项目打包和运行

在实际开发中，通常项目完成后不会将源代码公布给所有人，而是将项目和其依赖的组件组织成一个可执行文件分发到目标系统上运行或者交付给其他人使用，这个组织的过程也称为打包。项目打包后在其他环境中可以很方便地运行，Spring Boot 项目打包时通常会被创建为可执行的 JAR 包或 WAR 包，这两种包内部的文件结构不同，其运行的方式也不同。下面分别对这两种方式的打包和运行进行讲解。

1.5.1　打包为 JAR 包并运行

Spring Boot 应用内嵌了 Web 服务器，所以基于 Spring Boot 开发的 Web 应用也可以独立运行，无须部署到其他 Web 服务器中。下面以打包 chapter01 项目为例，将 Spring Boot 项目打包为可执行的 JAR 包并运行，具体操作如下。

1. 打包为可执行的 JAR 包

（1）添加 Maven 打包插件。SpringBoot 程序是基于 Maven 创建的，在对 Spring Boot 项目进行打包前，需要在项目 pom.xml 文件中加入 Maven 打包插件，Spring Boot 为项目打包提供了整合后的 Maven 打包插件 spring-boot-maven-plugin，可以直接使用，具体代码如下。

```
<build>
    <plugins>
        <!-- Maven 打包插件 -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

（2）使用 IDEA 进行打包。在 Maven 中提供了 package 打包指令，IDEA 中也提供了非常便捷的项目打包支持。在此选择在 IDEA 中对项目进行打包。在 IDEA 中单击右侧的“ Maven ”工具栏，会弹出 Maven 的操作界面，如图 1-20 所示。

在图 1-20 中，Lifecycle 下展示了 Maven 项目构建生命周期中常用的命令，选中对应的命令双击后可以快速执行该 Maven 命令。其中，clean 命令可以清除所有在构建过程中生成的文件，test 命令可以使用合适的单元测试框架来测试编译的源代码，package 命令可以完成项目编译、单元测试、打包功能。

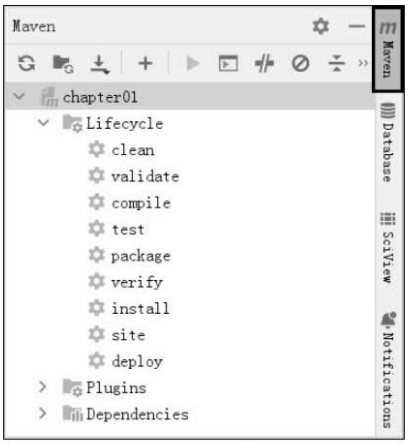


图1-20 Maven的操作界面

为了确保打包后的项目为最新编译的代码，并且不包含测试阶段的代码，可以在打包前先执行 clean 命令，在跳过测试阶段的模块后进行打包。首先在 Maven 操作界面中双击“ clean ”执行清除 target 操作，然后单击界面上方的⊘图标设置打包时跳过测试阶段的模块，最后双击“ package ”执行打包操作。执行打包操作后，会在控制台中输出打包的结果，如果打包成功会在项目的 target 文件夹下创建项目对应的可执行 JAR 包，如图 1-21 所示。

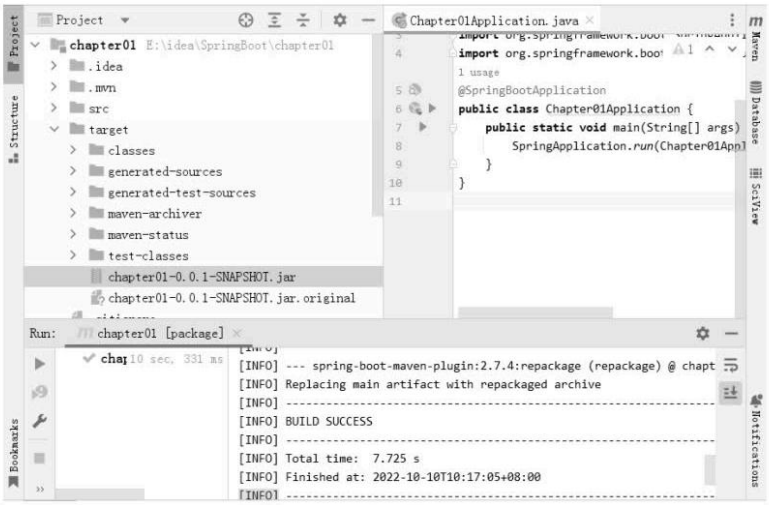


图1-21 打包结果（1）

从图 1-21 可以看出，控制台输出“ BUILD SUCCESS ”等信息，并且在项目的 target 文件夹下创建了以项目名称为开头命名的 JAR 包，说明成功将项目打包为可执行的 JAR 包。

2. 运行 JAR 包

一个可执行的 JAR 包可以由 Java 虚拟机（Java Virtual Machine，JVM） 直接执行而无须事先提取文件或者设置类路径。对此可以直接使用 Java 命令运行可执行的 JAR 包，运行 JAR 包的命令的语法格式如下。

```
java -jar JAR包名称
```

在上述语法中，JAR 包名称为包含 JAR 后缀的完整名称。

运行 JAR 包时可以选择在 IDEA 中或者在 cmd 窗口中执行，这两种方式没有什么区别，这里选择在 cmd 窗口中执行。在本地打开项目的 target 文件夹，如图 1-22 所示。

图1-22　target文件夹

从图 1-22 可以看出，打包的可执行 JAR 包存放在 target 文件夹下。

在图 1-22 的地址栏中输入“ cmd ”后按下“ Enter ”键，在当前文件夹路径下打开 cmd 窗口，并在弹出的 cmd 窗口中使用 Java 命令运行 chapter01-0.0.1-SNAPSHOT.jar，如图 1-23 所示。

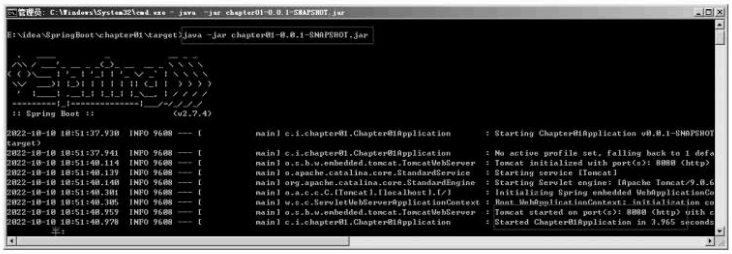


图1-23　运行JAR包

从图 1-23 可以看出，Spring Boot 项目打成的 JAR 包已经成功运行，并显示了默认的端口号 8080。需要注意的是，由于执行 Java 命令需要计算机中安装了 Java 的 JDK 环境，如果没有安装的话，执行后会报错。

JAR 包运行后，可以对项目进行访问，以测试打包后的效果，在浏览器中访问 http://localhost:8080/first，如图 1-24 所示。

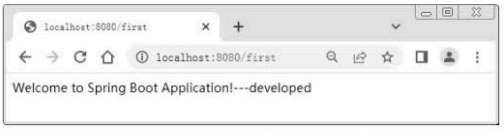


图1-24　程序测试效果（3）

从图 1-24 可以得出，运行 JAR 包后成功访问了项目中的资源。

1.5.2 打包为 WAR 包并运行

虽然通过 Spring Boot 内嵌的 Tomcat 可以将项目打成 JAR 包后直接运行，但每个 JAR 包中都会包含独立的 Web 容器，对服务器的使用消耗会比较大。当想要在一个 Web 容器中运行多个项目时，可以把每个项目打包成一个 WAR 包，然后部署到 Web 容器中运行。

下面以打包 chapter01-maven 项目为例，将 Spring Boot 项目打包为 WAR 包并运行，具体操作如下。

1. 打包为可执行的 WAR 包

（1）声明打包方式为 war。默认情况下创建的 Spring Boot 项目打包方式为 jar，要将项目打包为 WAR 包，需要在项目的 pom.xml 文件中声明当前项目的打包方式为 war。打开 chapter01-maven 项目的 pom.xml 文件，使用<packaging>标签声明项目打包方式为 war，具体代码如下。

```
<groupId>com.itheima</groupId>
<artifactId>chapter01-maven</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>
```

（2）排除内置 Tomcat。Spring Boot 为项目提供了内嵌的 Tomcat 服务器，使用外部的 Tomcat 时，需要在 pom.xml 文件中排除内置的 Tomcat，具体代码如下。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

在上述代码中，在 spring-boot-starter-web 的依赖中使用<exclusion>标签声明排除该启动器中的 Tomcat。

（3）添加 Tomcat 依赖。排除内置的 Tomcat 后，需要在 pom.xml 文件中手动添加 Tomcat 的依赖，以便在后续开发中使用对应的 API，具体代码如下。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <!--仅在编译和测试阶段使用，不会被打包-->
    <scope>provided</scope>
</dependency>
```

在上述代码中添加了 Tomcat 的依赖，由于对应 Tomcat 的依赖只需在编译和测试阶段使用，所以使用<scope>标签将该依赖的作用范围指定在编译和测试阶段，避免打包时将对应的依赖打包到 WAR 包中。

（4）添加插件。在项目的 pom.xml 文件中定义打包插件，以及项目打包后包的名称，具体代码如下。

```
<build>
```

```
    <finalName>springboot-war</finalName>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

在上述代码中，使用<finalName>标签用于指定项目打包后的名称；<plugin>标签用于指定打包插件。

（5）修改 Spring Boot 启动类。Spring Boot 启动项目的方式有很多种，使用外置 Tomcat 时，默认启动类需要继承 SpringBootServletInitializer 类，并重写 configure()方法。SpringBootServletInitializer 执行时，会通过重写的 configure()方法中的 SpringApplicationBuilder 实例构建并封装 SpringApplication 对象，并最终调用 SpringApplication 的 run()方法进行项目的启动。修改后的 Spring Boot 启动类如文件 1-8 所示。

文件 1-8　Chapter01MavenApplication.java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.SpringBootApplication;
3  import org.springframework.boot.builder.SpringApplicationBuilder;
4  import org.springframework.boot.web.servlet.ServletComponentScan;
5  import org.springframework.boot.web.servlet.
6         support.SpringBootServletInitializer;
7  @ServletComponentScan  // 开启基于注解方式的 Servlet 组件扫描支持
8  @SpringBootApplication
9  public class Chapter01MavenApplication  extends
10    SpringBootServletInitializer {
11    @Override
12    protected SpringApplicationBuilder configure(
13       SpringApplicationBuilder builder) {
14       return builder.sources(Chapter01MavenApplication.class);
15    }
16    public static void main(String[] args) {
17       SpringApplication.run(Chapter01MavenApplication.class, args);
18    }
19 }
```

在上述代码中，启动类 Chapter01MavenApplication 继承 SpringBootServletInitializer 类并重写 configure()方法，在 configure()方法中，sources(Chapter01 Maven Application.class)方法的参数必须是项目主程序的启动类。需要说明的是，为 Spring Boot 提供启动的 Servlet 初始化器 SpringBootServletInitializer 时，典型的做法就是让主程序启动类继承 SpringBootServletInitializer 类并实现 configure()方法；除此之外，还可以在项目中单独提供一个继承 SpringBootServletInitializer 的子类，并实现 configure()方法。

至此，将项目打包为 WAR 包的准备工作已经完成，下面参照 1.5.1 小节中使用 Maven 工具栏中的命令打包项目的过程，将 chapter01-maven 项目进行打包，打包后的结果如图 1-25 所示。

从图 1-25 可以看出，控制台输出“ BUILD SUCCESS ”等信息，并且在项目的 target 文件

夹下创建了项目名称为 springboot-web 的 WAR 包，说明成功将项目打包为可执行 WAR 包。

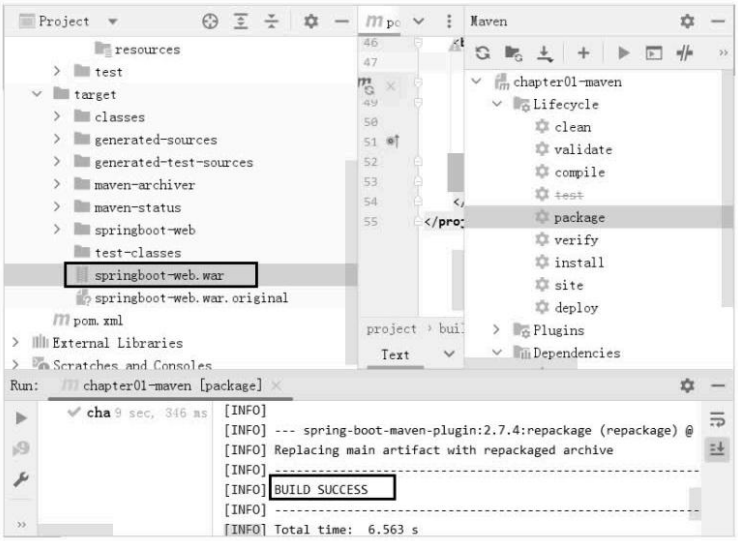


图1-25 打包结果（2）

2. 运行 WAR 包

将打包好的 WAR 包复制到本地 Tomcat 安装目录下的 webapps 文件夹中，在 cmd 窗口中执行 Tomcat 安装目录下 bin 目录中的 startup.bat 命令启动 Tomcat。Tomcat 启动后，执行效果如图 1-26 所示。

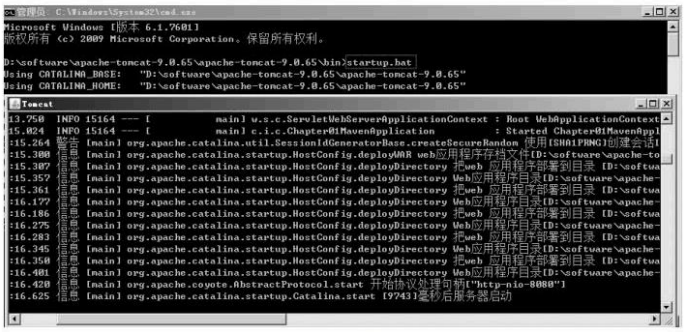


图1-26 启动Tomcat

从图 1-26 可以看出，Tomcat 启动成功。Tomcat 启动时会自动解压 Tomcat 安装目录下的 webapps 文件夹中的 WAR 包，并部署在 Tomcat 中，此时可以对项目进行访问。需要说明的是，对这种使用外部 Tomcat 部署的项目进行访问时，必须加上项目名称，即打包成 WAR 包后的项目全名，例如访问 chapter01-maven 项目映射路径为 first 的 Controller，其对应的请求地址为“ http://localhost:8080/springboot-web/first ”,在浏览器中的访问效果如图 1-27 所示。

图1-27 程序测试效果（4）

从图 1-27 可以得出，将打包的 WAR 包部署到外部 Tomcat 后，可以正常对项目的资源进行访问。

需要注意的是，Spring Boot 2.7.6 默认内嵌 Tomcat 的版本为 9.0.69，将指定版本的 Spring Boot 项目以 WAR 包形式部署到外部 Tomcat 中时，应尽量使用与 Spring Boot 项目匹配的 Tomcat 版本进行项目部署，否则在部署过程中可能出现异常。

多学一招：Tomcat 启动失败和控制台乱码

1. Tomcat 启动失败

在 cmd 窗口执行 startup.bat 命令启动 Tomcat 时，可能不能正常启动 Tomcat，并出现图 1-28 所示的提示信息。

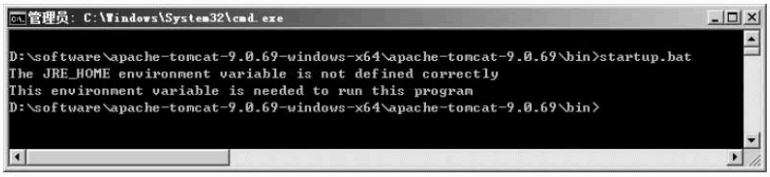


图1-28　Tomcat启动失败提示信息

从图 1-28 可以看出，控制台提示 JRE_HOME 环境变量没有正确定义。对此，可以在计算机的环境变量中新增一个名称为 JRE_HOME 的环境变量，变量值设置为 JDK 的安装路径即可。

2. Tomcat 控制台输出乱码

控制台输出乱码通常是编码不一致导致的，通过修改控制输出的编码即可解决乱码问题。打开 Tomcat 安装目录 conf 文件夹下的 logging.properties 文件，将文件中“java.util.logging.ConsoleHandler.encoding”的值修改为 GBK，如图 1-29 所示。

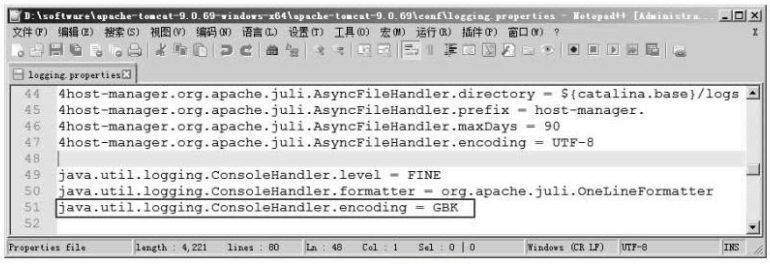


图1-29　修改控制台输出的编码

1.6　本章小结

本章主要对 Spring Boot 开发入门知识进行了讲解。首先讲解了 Spring Boot 概述；然后讲解了 Spring Boot 入门案例，并结合入门案例对 Spring Boot 的原理进行了解析；接着讲解了单元测试和热部署；最后讲解了 Spring Boot 项目的打包和运行。通过本章的学习，希望大家可以对 Spring Boot 有一个初步认识，为后续学习 Spring Boot 做好铺垫。

1.7 本章习题

一、填空题

1．Spring Boot 框架在开发过程中大量使用_____的思想来摆脱各种复杂的手动配置。

2．Spring Boot 2.7.6 官方声明支持的项目构建工具包括有_____和 Gradle。

3．@SpringBootApplication 注解内部包含的核心注解有_____、@EnableAutoConfiguration、@ComponentScan。

4．Spring Boot 启动类中调用 SpringApplication 类的_____方法来创建 Spring 容器。

5．Spring Boot 项目中，进行 Web 环境模拟测试包含_____和发送 Web 请求两个部分。

二、判断题

1．Spring Boot 是替代 Spring 框架的解决方案。（　　）

2．使用 Maven 方式构建 Spring Boot 项目必须处于联网状态，否则会创建失败。（　　）

3．Spring Boot 官方为所有场景开发的技术框架都提供了启动器。（　　）

4．DEFINED_PORT 为 webEnvironment 属性的默认值。（　　）

5．在 Spring Boot 项目中加入 spring-boot-devtools 热部署依赖启动器后重启项目即可生效。（　　）

三、选择题

1．下列选项中，关于 Spring Boot 概述错误的是（　　）。

A．使用 Spring Boot 开发程序时，几乎可以实现开箱即用。

B．Spring Boot 框架本身并不提供 Spring 框架的核心特性和扩展功能。

C．Spring Boot 是替代 Spring 框架的解决方案。

D．Spring Boot 集成了大量常用的第三方库配置。

2．下列选项中，关于 Spring Boot 自动装配时对应注解的作用描述错误的是（　　）。

A．@SpringBootConfiguration 标注当前类是一个配置类。

B．@EnableAutoConfiguration 可以关闭 Spring Boot 的自动配置。

C．@ComponentScan 注解是一个组件包扫描器。

D．被@SpringBootApplication 标注的类可以被扫描到 Spring 的 IoC 容器中。

3. 下列选项中，对于 Spring Boot 应用程序启动器描述错误的是（　　）。

A．spring-boot-starter-parent 为核心启动器，常被作为父依赖。

B．spring-boot-starter-web 使用 SpringMVC 构建 Web。

C．spring-boot-starter-test 提供 Logging 相关的日志功能。

D．spring-boot-starter-jdbc 结合 JDBC 和 HikariCP 连接池的启动器。

4．下列选项中，关于@SpringBootTest 注解 webEnvironment 属性的值描述错误的是(　　)。

A．MOCK 为 webEnvironment 属性的默认值。

B．DEFINED_PORT 加载一个 EmbeddedWebApplicationContext 并提供一个真正的 Servlet 环境。

C．RANDOM_PORT 会使用随机端口作为 Web 服务器端口。

D．NONE 会使用 SpringApplication 加载 ApplicationContext，但不启动 Web 环境。

5．下列选项中，关于 Spring Boot 项目以 WAR 包方式进行打包部署的说法错误的是（　　）。

A．要使用<packaging>标签将 Spring Boot 项目打包方式修改为 war。

B．使用<scope>out</scope>将该服务器声明为外部 out。

C．使用外置 Tomcat 时，默认启动类需要继承 SpringBootServletInitializer 类，并重写 configure()方法。

D．可以使用 IDEA 的 Maven 工具将 Spring Boot 项目打包为 WAR 包。

第2章

Spring Boot 配置

学习目标

★ 掌握 application.properties 配置文件，能够在 application.properties 配置文件中正确配置数据

★ 掌握 application.yml 配置文件，能够在 application.yml 配置文件中正确配置数据

★ 掌握@Value 注解，能够使用@Value 注解为 Bean 的属性绑定配置数据

★ 熟悉 Environment 对象，能够使用 Environment 对象获取全局配置文件中的属性

★ 掌握@ConfigurationProperties 注解，能够使用@ConfigurationProperties 注解为 Bean 的属性绑定配置数据

★ 了解@Value 和@ConfigurationProperties 对比分析，能够说出@Value 和@ConfigurationProperties 的主要区别

★ 掌握引入配置文件，能够使用@PropertySource 注解和@ImportResource 注解引入配置文件

★ 掌握定义配置类，能够使用@Configuration 注解定义配置类

★ 熟悉单一文件中配置 Profile，能够在单一文件中配置 Profile 以实现多环境配置

★ 掌握多文件中配置 Profile，能够在多文件中配置 Profile 以实现多环境配置

★ 熟悉@Profile 注解，能够正确使用@Profile 注解进行多环境配置

Spring Boot 极大地简化了 Spring 应用的开发，尤其是 Spring Boot 的自动配置功能，该功能使项目即使不进行任何配置，也能顺利运行。当用户想要根据自身需求覆盖 Spring Boot 的默认配置时，需要使用配置文件修改 Spring Boot 的默认配置。本章将对 Spring Boot 的配置进行讲解。

2.1 全局配置文件

全局配置文件能够对一些默认配置值进行修改。Spring Boot 默认使用的全局配置文件有 application.properties 和 application.yml，Spring Boot 启动时会自动读取这两个文件中的配置，如果文件中存在与默认自动配置相同的配置信息，则覆盖默认的配置信息。下面对全局配置文件进行讲解。

2.1.1 application.properties 配置文件

使用 Spring Initializr 方式构建 Spring Boot 项目时，会在 resource 目录下自动生成一个空的 application.properties 文件；使用 Maven 方式构建 Spring Boot 项目时，可以手动在 resource 目录下创建 application.properties 文件。

application.properties 文件中可以定义 Spring Boot 项目的相关属性，属性可采用键值对格式进行设置，表示形式为“ Key=Value ”，这些相关属性可以是系统属性、环境变量、命令参数等信息，也可以是自定义的属性。示例代码如下。

```
address=beijing
server.port=80
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

application.properties 文件中的属性支持多种类型，常见的有字面量、数组和集合，下面分别对这 3 种类型的属性的写法进行讲解。

1. 字面量类型属性

字面量是指单个的、不可拆分的值，例如：数字、字符串、布尔值等。在 application.properties 文件中配置字面量的属性时，直接将字面量作为 Value 写在键值对中即可，且默认情况下字符串是不需要使用单引号或双引号进行修饰的，示例代码如下。

```
address=beijing
age=13
```

在上述代码中，分别使用字面量 beijing 和 13 为属性 address 和 age 赋值。

如果需要配置的属性为对象的属性，可以通过“ 对象名.属性名 ”的方式指定属性的键。对象中可能包含多个属性，在 application.properties 文件中为对象的属性赋值时，一个属性对应一对键值对，示例代码如下。

```
user.username=lisi
user.age=18
```

在上述代码中，分别为 user 对象的 username 属性和 age 属性赋值。

2. 数组类型属性

在 application.properties 文件中配置数组类型属性时，可以将数组元素的值写在一行内，元素值之间使用逗号（,）间隔，也可以在多行分别根据索引赋值，示例代码如下。

```
# 方式一
user.hobby=swim,travel,cook
# 方式二
user.hobby[0]=swim2
user.hobby[1]=travel2
user.hobby[2]=cook2
```

在上述代码中，使用两种方式为 user 对象的属性 hobby 数组赋值。

3. 集合类型属性

在 application.properties 文件中也可以配置集合类型的属性，下面分别演示配置 List、Set、Map 的集合类型属性，示例代码如下。

```
# 配置 List:方式一
#user.subject=Chinese,English,Math
# 配置 List:方式二
user.subject[0]=Chinese
```

```
user.subject[1]=English
user.subject[2]=Math
# 配置 Set
user.salary=120,230
# 配置 Map：方式一
user.order.1001=cookie
user.order.1002=cake
# 配置 Map：方式二
user.order[1001]=cookie
user.order[1002]=cake
```

在上述代码中，分别为配置了 List、Set、Map 集合类型的属性。其中，配置 List 类型的属性时，可以在一行内配置一组按次序排列的值，也可以在多行内根据索引进行配置；配置 Set 类型的属性时，与配置 List 的行内式格式一致；配置 Map 时，可以通过“属性名.键”的方式配置，也可以通过“属性名[键]”的方式配置。

2.1.2 application.yml 配置文件

application.yml 配置文件是使用 YAML 编写的文件，YAML 是“YAML Ain't Markup Language”的递归缩写。YAML 通常用于表示数据结构和配置信息，它使用缩进和外观依赖的方式表示层级关系，使得配置文件和数据结构的表达相对简洁和易于阅读。YAML 支持的数据包括列表、键值对和字符串、数字等。YAML 文件的后缀名为.yml 或.yaml，编写时需要遵循以下规则。

- 使用缩进表示层级关系。
- 缩进时不允许使用“Tab”键，只允许使用空格。
- 缩进的空格数不重要，但同级元素必须左侧对齐。
- 对大小写敏感。

application.yml 与 application.properties 一样，可以在 Spring Boot 启动时被自动读取。下面在 application.yml 中演示配置常见数据类型的属性。

1. 字面量类型属性

YAML 中，使用“Key: Value”的形式表示一对键值对，其中 Value 前面有一个英文空格，并且该空格不能省略。在配置字面量类型的属性时，直接将字面量作为 Value 直接写在键值对中即可，且默认情况下字符串是不需要使用单引号或双引号的。示例代码如下。

```
address: beijing
age: 13
```

在上述代码中，分别使用字面量 beijing 和 13 为属性 address 和 age 赋值。

如果需要配置的属性为对象的属性，配置的方式有缩进式和行内式两种，示例代码如下。

```
# 缩进式
consumer:
  username: lisi
  age: 18
# 行内式
consumer: {username: lisi,age: 18}
```

在上述代码中，使用两种方式为 consumer 对象的 username 属性和 age 属性赋值。其中

使用缩进式时，用缩进表示对象与属性的层级关系，在 IDEA 中的 application.yml 中编辑属性时，在冒号（:）后按“ Enter ”键会自动进行缩进；行内式使用大括号包含对象的属性，属性之间使用逗号（,）间隔。

2. 数组类型和单列集合属性

当 YAML 配置文件中配置的属性为数组类型或单列集合时，也可以使用缩进式写法和行内式写法。其中，缩进式写法示例如下。

```
consumer:
  hobby:
    - play
    - read
    - sleep
```

在上述代码中，在 YAML 配置文件中通过缩进式写法为 consumer 对象数组类型或单列集合的 hobby 属性赋值 play、read 和 sleep。其中，使用“ - 属性值 ”指定元素值，“ - ”和属性值之间使用英文空格间隔。

在 YAML 配置文件中，还可以将缩进式写法简化为行内式写法，示例代码如下。

```
consumer:
  hobby: [play,read,sleep]
```

通过上述示例对比发现，YAML 配置文件的行内式写法更加简明、方便。另外，包含属性值的中括号“ [] ”可以省略。

3. Map 集合属性

当 YAML 配置文件中配置的属性为 Map 集合时，可以使用缩进式写法和行内式写法，其中，缩进式写法的示例代码如下。

```
consumer:
  order:
    1001: cookie
    1002: cake
```

对应的行内式写法示例代码如下。

```
consumer:
  order: {1001: cookie,1002: cake}
```

在上述代码中，order 为 Map 类型的属性名，1001 和 1002 为 Map 的键，cookie 和 cake 为 Map 中键对应的值。

多学一招：默认配置文件

Spring Boot 项目将 application.properties 或 application.yml 作为项目的默认配置文件，在默认配置文件中，用户可以定义或编辑项目的全局配置。Spring Boot 项目中可以存在多个 application.properties 或 application.yml，Spring Boot 启动时会扫描以下 5 个位置的 application.properties 和 application.yml 文件，并将扫描到的文件作为 Spring Boot 的默认配置文件。

① file:./config/*/。
② file:./config/。
③ file:./。
④ classpath:/config/。

⑤ classpath:/。

上述位置中，“file:”指当前项目根目录；“classpath:”指当前项目的类路径，即 src.main.java 和 src.main.resources 路径，以及第三方 JAR 包的根路径。

上述 5 个位置下如果存在 application.properties 和 application.yml 文件，在项目启动时就会被加载。加载多个 application.properties 或 application.yml 文件时，文件中的配置会根据文件所处的位置划分优先级，优先级规则如下：

- 上述位置 1～位置 5 的优先级依次降低，序号越小优先级越高。
- 位于相同位置的 application.properties 的优先级高于 application.yml，application.yml 的优先级高于 application.yaml。
- 存在相同的配置内容时，高优先级的内容会覆盖低优先级的内容。
- 存在不同的配置内容时，高优先级和低优先级的配置内容取并集。

2.2 配置绑定

使用 Spring Boot 全局配置文件配置属性时，如果配置的属性是 Spring Boot 内置的属性（如服务端口 server.port），那么 Spring Boot 会自动扫描并读取配置文件中的属性值并覆盖原有默认的属性值。如果配置的属性是用户自定义的属性，可以通过 Java 代码去读取该配置属性，并且把属性绑定到 Bean。在 Spring Boot 项目中可以通过 @Value、Environment 对象和@ConfigurationProperties 对配置属性进行绑定，下面分别对这三种方式实现配置绑定进行讲解。

2.2.1 @Value 注解

@Value 注解是由 Spring 框架提供的，Spring Boot 框架从 Spring 框架中对@Value 注解进行了默认继承，通过@Value 可以将配置文件中的属性绑定到 Bean 对象对应的属性。使用 @Value 注入属性的示例代码如下。

```
@Component
public class Person {
@Value("${person.id}")
   private int id;
}
```

在上述代码中，在类上使用@Component 进行标注，是因为如果想要将配置数据绑定到 Bean 对象，首先需要确保 Bean 对象在 IoC 容器中。@Value 将配置文件中属性 person.id 的值动态注入到 id 属性。

下面通过案例演示在 Spring Boot 项目中使用@Value 绑定全局配置文件中的数据，具体如下。

（1）创建实体类。在 IDEA 中创建一个 Spring Boot 项目，在项目的 java 文件夹下创建类包 com.itheima.domain，并在该类包下创建一个消费者实体类 Consumer，在该类上使用 @Component 进行标注，并在属性上使用@Value 注解注入配置文件中的属性，具体如文件 2-1 所示。

文件 2-1 Consumer.java

```
1 import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.stereotype.Component;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
@Component
public class Consumer {
    @Value("${consumer.username}")
    private String username;
    @Value("${consumer.age}")
    private  int age;
    @Value("#{'${consumer.hobby}'.split(',')}")
    private  String[] hobby;
    @Value("${consumer.subject}")
    private List subject;
   //……getter 方法，以及 toString()方法
}
```

在文件 2-1 中，Consumer 类使用@Value 注解读取和注入了配置文件的属性值，在每一个需要注入配置信息的属性上使用@Value 注解标注，并提供属性的 getter()方法。

（2）添加配置信息。在项目的 resource 文件夹下创建配置文件 application.yml，在配置文件中添加属性信息，具体如文件 2-2 所示。

文件 2-2　application.yml

```
consumer:
  username: lisi
  age: 23
  hobby: sing,read,sleep
  subject: 100,150
```

（3）创建测试类。在项目 test 文件夹下，创建类包 com.itheima，在该包下创建测试类 Chapter02ApplicationTests，在该测试类中注入 Consumer 对象，并新增一个测试方法进行输出测试，具体如文件 2-3 所示。

文件 2-3　Chapter02ApplicationTests.java

```
import com.itheima.domain.Consumer;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
@SpringBootTest
class Chapter02ApplicationTests {
    @Autowired
    private Consumer consumer;
    @Test
    void wiredTest() {
        System.out.println(consumer);
    }
}
```

在上述代码中，第 7～8 行代码使用@Autowired 注解将 Spring 容器中的 Consumer 实体类 Bean 注入到 Consumer 对象；第 10～12 行代码的测试方法 wiredTest()中，将 Consumer 对象输出到控制台。

（4）测试程序效果。运行测试方法 wiredTest()，控制台输出效果如图 2-1 所示。

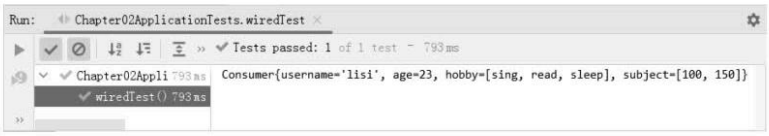


图2-1 wiredTest()方法运行结果

从图 2-1 可以看出，测试方法 wiredTest()运行成功，同时在控制台正确输出了 Consumer 对象，其中对象的属性值与配置文件中的属性值一致，说明使用@Value 注解成功将配置文件的属性绑定到 Bean。

需要说明的是，@Value 注解对 Map 集合的属性注入支持效果不佳，如果使用@Value 对 Map 集合的数据进行配置绑定，需要将配置文件中 Map 的数据中对应的 Vlaue 使用双引号进行包裹，否则解析时会失败。示例代码如下。

```
order: "{Key: 'Value'}"
```

2.2.2 Environment 对象

使用@Value 注解时，将该注解标注在 Spring 管控的 Bean 的属性名上方，就可以将某个数据绑定到 Bean 对象的属性。当 Bean 的属性比较多且这些属性都需要绑定配置的数据时，操作起来就比较烦琐。为此，Spring Boot 提供了一个对象 Environment，项目启动时能够将配置文件中的所有数据都封装到该对象中，这样就不需要手动对配置数据进行绑定。

使用 Environment 对象获取配置文件的数据时，不需要再提供其他实体类，下面对 Environment 对象的使用进行演示。

在文件 2-3 的 Chapter02ApplicationTests 类中，通过@Autowired 注入 Environment 对象，并新增测试方法 evnTest()，在测试方式中通过 Environment 对象获取配置文件中的属性，具体代码如下。

```
@Autowired
private Environment env;
@Test
void evnTest() {
    System.out.println("consumer.username="+
env.getProperty("consumer.username"));
    System.out.println("consumer.age="+
env.getProperty("consumer.age"));
    System.out.println("consumer.hobby="+
env.getProperty("consumer.hobby"));
    System.out.println("consumer.subject="+
env.getProperty("consumer.subject"));
}
```

上述代码的 evnTest()方法中，使用 Environment 对象的 getProperty()方法获取封装到 Environment 对象中的配置文件的数据，该方法的参数为配置数据键的名称。

运行测试方法 evnTest()，控制台输出效果如图 2-2 所示。

从图 2-2 可以看出，测试方法 evnTest()运行成功，同时在控制台输出配置文件中对应数据的信息。

图2-2 evnTest()方法运行结果

2.2.3 @ConfigurationProperties 注解

Java 是面向对象的语言，很多情况下，人们习惯将具有相同特性的一组数据封装到一个对象中，Spring Boot 中就提供了这样的注解。Spring Boot 的@ConfigurationProperties 注解可以将配置文件中的一组配置数据同时绑定到 Bean 中。

下面通过案例演示在 Spring Boot 项目中使用@ConfigurationProperties 注解绑定全局配置文件中的数据，具体如下。

（1）修改实体类属性的绑定方式。在文件 2-1 的 Consumer 类上使用@ConfigurationProperties 注解进行标注，并且去除属性上方标注的@Value 注解，具体代码如下。

```
@Component
@ConfigurationProperties(prefix = "consumer")
public class Consumer {
    private String username;
    private int age;
    private String[] hobby;
    private List subject;
   //……setter/getter 方法，以及 toString()方法
}
```

在上述代码中，@ConfigurationProperties 注解的 prefix 属性用于指定绑定配置文件中属性的前缀，Consumer 类中的属性名需要与绑定的配置文件中的属性名保持一致。

（2）新增测试方法。在文件 2-3 的 Chapter02ApplicationTests 中新增测试方法 confTest()，在测试方式中输出 Consumer 对象，具体代码如下。

```
@Test
void confTest() {
    System.out.println(consumer);
}
```

（3）测试程序效果。运行测试方法 confTest()，控制台输出效果如图 2-3 所示。

图2-3 confTest()方法运行结果

从图 2-3 可以看出，测试方法 confTest()运行成功，同时在控制台输出配置文件中对应数据的信息。说明使用@ConfigurationProperties 注解成功将配置文件的属性绑定到 Bean。

2.2.4 @Value 和@ConfigurationProperties 对比分析

通过前面的学习可知，@Value 注解和@ConfigurationProperties 注解都可以对配置文件中的属性进行绑定，但两者在使用过程中还是有一些差异的。软件开发人员进行技术选型时，应秉承高度的责任感，以问题为导向，根据实际应用场景选择更适合的技术，并制定合理的解决方案，在复杂的业务环境下科学客观地做好技术选型。

为了进一步了解两者的不同，下面对@Value 注解和@ConfigurationProperties 注解进行对比分析，具体如表 2-1 所示。

表 2-1 @Value 注解和@ConfigurationProperties 注解对比分析

对比项	@Value	@ConfigurationProperties
底层框架	Spring	Spring Boot
功能	单个注入配置文件中的属性	批量注入配置文件中的属性
为属性设置 setter 方法	不需要	需要
复杂类型属性注入	不支持	支持
松散绑定	不支持	支持
JSR303 数据校验	不支持	支持
SpEL 表达式	支持	不支持

关于表 2-1 中@Value 注解和@ConfigurationProperties 注解的对比分析，具体说明如下。

1. 底层框架

@Value 注解是由 Spring 框架提供的，只不过 Spring Boot 框架对 Spring 进行了默认支持，所以也可以使用@Value 注解的相关功能；@ConfigurationProperties 注解是由 Spring Boot 框架提供的。

2. 功能

@Value 注解只在需要注入属性值的单个属性上进行注入配置；@ConfigurationProperties 注解主要用于将配置文件中某一类属性整体批量读取并注入到 Bean 的属性中。

3. 为属性设置 setter 方法

@Value 不需要为属性设置 setter 方法，该注解会先通过表达式读取配置文件中指定的属性值，然后自动注入到下方的 Bean 属性上。如果读取的配置文件属性为空，进行属性注入时程序会报错。

@ConfigurationProperties 注解进行配置文件属性注入时，还必须为每一个属性设置 setter 方法。如果配置文件中没有配置属性值，则会自动将对应的 Bean 属性设置为空。

4. 复杂类型属性注入

@Value 在注入复杂数据类型配置数据时，会无法解析，导致注入失败；@ConfigurationProperties 注解支持任意数据类型的属性注入。

5. 松散绑定

@ConfigurationProperties 注解进行配置文件属性值注入时，支持松散绑定语法。例如 Person 类有一个字符串类型的属性 firstName，可以绑定配置文件中的以下属性。

```
person.firstName=james        // 标准写法，对应 Person 类属性名
person.first-name=james       // 使用横线-分隔多个单词
```

```
person.first_name=james        // 使用下画线_分隔多个单词
PERSON.FIRST_NAME=james        // 使用大小写格式，推荐常量属性配置
```

上述示例列举了@ConfigurationProperties 注解进行配置文件属性值注入时，支持的配置文件语法，属于松散绑定语法，而@Value 注解不支持此功能。

6. JSR303 数据校验

JSR303 数据校验的主要作用是校验配置文件中注入到对应 Bean 属性的值是否符合相关值的规则，@ConfigurationProperties 注解进行配置文件属性值注入时，支持 JSR303 数据校验，示例代码如下。

```
@Component
@ConfigurationProperties(prefix = "person")
@Validated      // 引入 Spring 框架支持的数据校验规则
public class Example {
    @Email   // 对属性进行规则匹配
    private String email;
    public void setEmail(String email) {
        this.email = email;
    }
}
```

在上述代码中，使用@ConfigurationProperties 注解注入配置文件属性值时，在实体类 Example 上引入@Validated 注解用于数据校验，在属性 email 上引入@Email 注解进行对应规则校验。如果注入的配置文件属性值不符合相关校验规则，程序会自动报错。@Value 注解不支持 JSR303 数据校验功能。

7. SpEL 表达式

使用@Value 注解注入配置文件属性时，支持 SpEL 表达式语法，即“ #{xx} ”。例如 Person 类有一个整数类型的属性 id，可直接使用 SpEL 表达式进行属性注入，示例代码如下。

```
@Value("#{5*2}")   // 使用@Value 注解的 SpEL 表达式直接为属性注入值
private int id;
```

在上述代码中，列举了在不使用配置文件的情况下，可以直接使用@Value 注解支持的 SpEL 表达式进行 Bean 的属性值注入，而@ConfigurationProperties 注解不支持此功能。

上述对@Value 和@ConfigurationProperties 两种注解进行配置文件属性值注入的主要区别进行了对比分析，但两者之间并没有明显的优劣之分，它们只是适合的应用场景不同而已，不同场景下的使用推荐如下。

- 如果只是针对某一个业务需求，要引入配置文件中的个别属性值，推荐使用@Value 注解。
- 如果针对某个 Java Bean 类，需要批量注入属性值，则推荐使用@ConfigurationProperties 注解。

2.3 引入配置文件和定义配置类

虽然 Spring Boot 免除了项目中大部分的手动配置，对于一些特定情况，可以通过修改全局配置文件以适应具体的开发或生产环境，但是有时候项目中不可避免地要使用默认配置文件之外的配置信息，这个时候就需要手动引入配置文件或定义配置类。下面分别对在

Spring Boot 项目中引入配置文件和定义配置类进行讲解。

2.3.1 引入配置文件

Spring Boot 项目中引入的配置文件通常有两类，第一类为 YAML 或 properties 的属性配置文件；第二类为 XML 配置文件。一般第一类配置文件可以使用@PropertySource 引入，第二类配置文件可以使用@ImportResource 引入，下面分别对使用这两个注解引入配置文件进行讲解。

1. 使用@PropertySource 引入属性配置文件

Spring Boot 默认会从全局配置文件 application.yml 或 application.properties 中读取配置信息，如果全部的配置都写在全局配置文件中，那么这个配置文件会十分臃肿且难以维护，因此，可以将一些自定义配置按照不同模块提取出来，放到自定义的配置文件中，然后在项目中引入该配置文件即可。

对于这种加载自定义配置文件的需求 ,可以使用@PropertySource 注解来实现。@PropertySource 注解标注在类上，使用时需要指定引入的配置文件的位置和名称。如果需要将自定义配置文件中的属性值注入到对应类的属性中，可以使用@ConfigurationProperties 或者@Value 注解进行注入。

下面通过案例演示在 Spring Boot 项目中使用@PropertySource 引入属性配置文件，具体如下。

（1）创建配置文件。在项目 chapter02 的 resources 目录下创建自定义配置文件 user.properties，在该配置文件中编写需要设置的属性，具体如文件 2-4 所示。

文件 2-4 user.properties

```
id=1001
nickname=wangwu
```

（2）创建实体类。在 com.itheima.domain 包下创建用户实体类 User，在类上使用@PropertySource 引入配置文件 user.properties，并使用@ConfigurationProperties 注解将配置文件中的属性绑定到类的属性上，具体如文件 2-5 所示。

文件 2-5 User.java

```
1  import org.springframework.boot.context.
2          properties.ConfigurationProperties;
3  import org.springframework.context.annotation.PropertySource;
4  import org.springframework.stereotype.Component;
5  @Component
6  @PropertySource("classpath:user.properties")
7  @ConfigurationProperties
8  public class User {
9     private String id;
10    private String nickname;
11    //……setter/getter 方法，以及 toString()方法
12 }
```

在上述代码中，第 6 行代码指定引入 classpath 下的 user.properties 配置文件；第 7 行代码将引入配置文件中的配置数据绑定到类的属性上。

（3）新增测试方法。在文件 2-3 的 Chapter02ApplicationTests 类中注入 User 对象，并新

增测试方法 propTest()，在测试方式中输出 User 对象，具体代码如下。

```
@Autowired
private User user;
@Test
void propTest() {
    System.out.println(user);
}
```

（4）测试程序效果。运行测试方法 propTest()，控制台输出效果如图 2-4 所示。

图2-4　propTest()方法运行结果

从图 2-4 可以看出，测试方法 propTest()运行成功，同时在控制台输出 user.properties 配置文件中对应数据的信息，说明使用@PropertySource 成功引入了属性配置文件。

2. 使用@ImportResource 引入 XML 配置文件

传统 Spring 框架大多采用 XML 文件作为配置文件，但 Spring Boot 推荐使用 Java 配置类进行配置，Spring Boot 默认不能自动识别 XML 配置文件，想让 Spring 的 XML 配置文件生效，可以使用@ImportResource 注解加载 XML 配置文件。

@ImportResource 注解标注在一个配置类上，使用时需要指定引入 XML 配置文件的路径和名称。下面通过案例演示在 Spring Boot 项目中使用@ImportResource 引入 XML 配置文件，具体如下。

（1）创建组件类。在 com.itheima.service 包下创建类 MyService，在类中定义方法用于后续测试，具体如文件 2-6 所示。

文件 2-6　MyService.java

```
1 public class MyService {
2     public void getById(String id){
3         System.out.println(id);
4     }
5 }
```

（2）创建 XML 配置文件。在 resources 文件夹下创建配置文件，在该配置文件中声明 Bean，具体如文件 2-7 所示。

文件 2-7　beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="myService" class="com.itheima.service.MyService" />
</beans>
```

在上述代码中，使用 Spring 框架的 XML 方式声明 Bean。

（3）添加@ImportResource 注解。编写完 Spring 的 XML 配置文件后，Spring Boot 默认不会自动引入，为了保证 XML 配置文件生效，需要在项目启动类 Chapter02Application 中添加

@ImportResource 注解来指定 XML 文件的位置，具体如文件 2-8 所示。

文件 2-8 Chapter02Application.java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.SpringBootApplication;
3  import org.springframework.context.annotation.ImportResource;
4  @SpringBootApplication
5  @ImportResource("classpath:beans.xml")
6  public class Chapter02Application {
7      public static void main(String[] args) {
8          SpringApplication.run(Chapter02Application.class, args);
9      }
10 }
```

在上述代码中，第 5 行代码使用@ImportResource 注解指定引入 classpath 路径下的 beans.xml 文件。

（4）新增测试方法。在文件 2-3 的 Chapter02ApplicationTests 类中注入 MyService 对象，并新增测试方法 beanTest()，在测试方式中使用 MyService 对象调用 getById()方法，具体代码如下。

```
@Autowired
private MyService myService;
@Test
void beanTest() {
    myService.getById("18");
}
```

（5）测试程序效果。运行测试方法 beanTest()，控制台输出效果如图 2-5 所示。

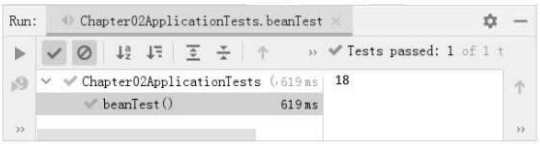


图2-5 beanTest()方法运行结果（1）

从图 2-5 可以看出，测试方法 beanTest()运行成功，同时在控制台输出调用 getById()方法的结果，说明使用@ImportResource 成功引入了 XML 配置文件。

2.3.2 定义配置类

虽然 Spring Boot 可以使用@ImportResource 注解来引入 XML 配置文件，但使用 Spring Boot 开发项目大多是为了减少配置，以便快速开发。当自动配置不能满足需求的时候，通常会通过配置类来对自定义 Bean 进行 Bean 容器的装配工作。添加@Configuration 注解的类称为配置类，Spring Boot 推荐使用配置类替代原来的配置文件。软件开发人员在使用 Spring Boot 时，要保持乐学善学的态度，提高终身学习的意识，不断认识和理解新知识的价值，掌握更适合当前框架的技术。

当定义一个配置类后，需要在类中的方法中使用@Bean 注解进行组件配置，将方法的返回对象注入到 Spring 容器中。组件名称默认使用的是方法名，也可以使用@Bean 注解的 name 或 value 属性自定义组件的名称。引入配置类时，只需让配置类被 Spring Boot 自动扫描识别即可，该配置类中返回的组件会自动添加到 Spring 容器中。

下面通过案例演示在 Spring Boot 项目中定义配置类，具体如下。

（1）创建配置类。在 com.itheima.config 包下创建 MyConfig 类，并使用@Configuration 注解将该类声明为一个配置类，具体如文件 2-9 所示。

文件 2-9　MyConfig.java

```
1  import com.itheima.service.MyService;
2  import org.springframework.context.annotation.Bean;
3  import org.springframework.context.annotation.Configuration;
4  @Configuration
5  public class MyConfig {
6      @Bean
7      public MyService myService(){
8          return  new MyService();
9      }
10 }
```

在上述代码中，第 6～9 行代码通过@Bean 注解将 MyService 对象添加到 Spring 容器中。

（2）测试程序效果。本案例使用@Configuration 和@Bean 将 MyService 对象添加到 Spring 容器中，可以将文件 2-8 中项目启动类 Chapter02Application 上添加的@ImportResource 注解注释掉，运行测试方法 beanTest()，控制台输出效果如图 2-6 所示。

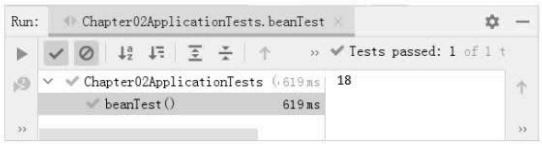


图2-6　beanTest()方法运行结果（2）

从图 2-6 可以看出，测试方法 beanTest()运行成功，同时在控制台输出调用 getById()方法的结果，说明定义的配置类成功替代了之前的 Spring 配置文件。

2.4　Profile

在实际开发中，根据项目的开发进度，项目经常需要在不同的部署环境间切换，常见部署的环境有开发环境、测试环境、生产环境。不同环境使用的配置信息往往不同，而且项目的配置信息往往有很多，如果每次变更项目部署的环境时，都采用手动方式更改配置信息会很麻烦。针对这种情况，在 Spring Boot 中可以使用 Profile 解决这类问题，Profile 使 Spring Boot 可以针对不同的环境提供不同的配置。在 Spring Boot 中可以将 Profile 配置在单一文件中和多个文件中，也可以通过@Profile 注解指定 Bean 的生效环境。下面对 Profile 的使用进行讲解。

2.4.1　单一文件中配置 Profile

Spring Boot 中可以在配置文件使用 spring.config.activate.on-profile 指定 Profile 的名称，使用 spring.profiles.active 指定激活哪个 Profile，如果需要激活多个 Profile，Profile 名称之间使用逗号间隔即可。每个 Profile 中的配置信息都对应于一个部署环境，在单一 YAML 文件配置多个 Profile 时，可以通过三个短横线号（---）将不同的 Profile 分隔开。下面通过案

例演示在单一文件中配置 Profile，具体如下。

（1）配置 Profile。在项目 chapter02 的 application.yml 配置文件中配置 3 个 Profile，名称分别为 dev、test、pro，表示开发环境、测试环境和生产环境，具体配置信息如文件 2-10 所示。

文件 2-10 application.yml

```
# 指定激活的 profiles
spring:
  profiles:
    active: dev
---
# 开发环境
spring:
  config:
    activate:
      on-profile: dev
server:
  port: 80
---
# 测试环境
spring:
  config:
    activate:
      on-profile: test
server:
  port: 81
---
# 生产环境
spring:
  config:
    activate:
      on-profile: pro
server:
  port: 82
```

在上述代码中，使用“---”将配置信息分为 4 个部分。其中，第 2～4 行代码指定项目启动时激活的 Profile 为 dev；第 7～28 行代码中，每个 Profile 都配置了项目的服务端口号。

（2）创建控制器类。在 com.itheima.controller 包下创建控制器类 DefaultController，在类中注入 Environment 对象，并定义获取项目服务端口的方法，具体如文件 2-11 所示。

文件 2-11 DefaultController.java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.env.Environment;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class DefaultController {
   @Autowired
   private  Environment env;
    @RequestMapping("/getPort")
    public String getPort(){
```

```
11        String port = env.getProperty("server.port");
12        System.out.println("server.port:"+port);
13        return "server.port:"+port;
14    }
15 }
```

在上述代码中，第 11 行代码用于获取配置文件中的 server.port 属性。

（3）测试程序效果。启动项目，控制台输出如图 2-7 所示。

图2-7　控制台输出（1）

从图 2-7 可以看出，控制台输出信息中提示项目启动时激活的 profile 为 dev，Tomcat 的端口号为 80。

在浏览器中访问 http://localhost/getPort，效果如图 2-8 所示。

从图 2-8 可以看出，页面返回的端口号为 80，说明启动的环境为开发环境。

如果将文件 2-11 中激活的 Profile 修改为 pro，再次启动项目时，项目控制台输出如图 2-9 所示。

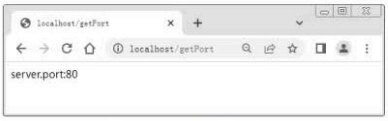


图2-8　访问效果（1）

图2-9　控制台输出（2）

从图 2-9 可以看出，控制台输出信息中提示启动时激活的 profile 为 pro，Tomcat 的端口号为 82，说明如果要切换环境，只需修改激活的 Profile 名称即可，启动不同的 Profile，项目会加载对应 Profile 中的配置属性。

至此，在单一文件中配置 Profile 已经完成。

2.4.2　多文件中配置 Profile

实际开发中，项目中通常会包含多个组件或框架，如果将所有的配置信息都放在一个配置文件中，尤其是配置的部署环境都不一样时，配置文件会非常臃肿，不便于维护。针对此情况，可以将一个配置文件拆分成多个配置文件。拆分后，可在不同的配置文件中配置不同环境的信息，并在主配置文件中指定激活的 Profile。

拆分出的配置文件的名称格式为 application-{profile}.yml 或 application-{profile}.properties，其中{profile}对应具体环境标示的 Profile 名称。例如，YAML 格式的开发环境、测试环境和生产环境配置文件命名如下。

```
application-dev.yml      // 开发环境配置文件
```